DESIGN AND DEVELOPMENT OF AN AUTOMATED RISK ASSESSMENT AND GOVERNANCE FRAMEWORK FOR SHADOW AI AGENTS IN CORPORATE NETWORKS

A Research Dissertation Submitted to the Faculty of Computer Science and Engineering
in Partial Fulfillment of the Requirements for the Award of
Bachelor of Science (Hons) in Cybersecurity

By

S.H.I. Madhushan
Student ID: 14504

Supervised by:
Mr. Sahan Weerasinghe

Department of Computer Networks and Cyber Security
KIU University, Sri Lanka
2026

Declaration

I hereby declare that this dissertation titled 'Design and Development of an Automated Risk Assessment and Governance Framework for Shadow AI Agents in Corporate Networks' is my own original work conducted under the supervision of Mr. Sahan Weerasinghe at KIU University, Sri Lanka.

This document has not been submitted previously, in whole or in part, for the award of any degree or diploma at this or any other tertiary education institution. All secondary information, literature references, code modules and conceptual models used herein have been duly acknowledged in accordance with standard IEEE referencing protocols.

Student Signature: _____ **Date:** _____

Supervisor Signature: _____ **Date:** _____

Abstract

The rapid integration of Artificial Intelligence (AI) and Large Language Models (LLMs) into enterprise workflows by 2026 has catalyzed unprecedented operational efficiency but has concurrently introduced a profound security and governance challenge: Shadow AI. This phenomenon involves the unsanctioned deployment and interaction with AI endpoints and autonomous agentic workflows by corporate employees, operating entirely outside the monitoring and control of formal IT security teams.

Unlike traditional Shadow IT, Shadow AI is inherently conversational, persistent and semantically rich. Natural-language prompts transmitted to external AI providers frequently contain confidential corporate intellectual property, such as proprietary source code, electronic health records (EHR), internal network topographies and strategic financial projections. Legacy perimeter security architecture specifically Next Generation Firewalls (NGFW) and signature-based Data Leakage Prevention (DLP) tools remain fundamentally '**context-blind**'. They are structurally incapable of evaluating natural language intent hidden within encrypted TLS 1.3 HTTPS traffic flows.

To address this governance and visibility crisis, this research presents ContextGuard, a high-fidelity prototype framework for automated risk assessment and governance of Shadow AI agents in enterprise network environments. Developed and tested within a simulated corporate network infrastructure ('NexusCorp Enterprise'), ContextGuard implements a 4-tier decoupled pipeline consisting of:

- **Active MITM Ingestion & Vendor Payload Parsing:** Using mitmproxy v10 and custom Double-Plane URL Decoding Engine to capture outbound TLS POST flows and unquote complex vendor encodings.
- **Dual-Pronged Inspection Engine:** Executing parallel Heuristic Behavioral Analysis (evaluating Inter-Arrival Time variance and packet distributions to isolate autonomous bot scripts) and NLP-driven Semantic Inspection (tokenizing prompts against an in-memory database of 15 Master CSV Corporate Asset datasets using NLTK and TF-IDF).
- **Weighted Risk Scoring Engine (WRSE):** Formulating a multi-factor mathematical scoring model that aggregates Data Sensitivity (S), Destination Trust (D) and User Profile Weight (U) into a normalized risk score (0-100).

- **Zero - Trust Security & SIEM Dashboard:** Deploying an interactive Streamlit SOC dashboard protected by WAF regex sanitization, brute-force lockout defenses and server-side UUID session persistence.

Empirical host machine penetration experiments demonstrated high detection accuracy (>80% True Positive Rate) for active data exfiltration attempts. While current technical limitations restrict content inspection on binary file uploads (PDF, DOCX, image binaries), a detailed OCR integration roadmap is formulated. ContextGuard confirms the technical and architectural feasibility of dynamic, context-aware AI governance for enterprise Zero-Trust security.

Keywords: Shadow AI, Cybersecurity Governance, Weighted Risk Scoring Engine (WRSE), Behavioral Anomaly Detection Data Leakage Prevention (DLP), Natural Language Processing, Zero-Trust Architecture.

Acknowledgement

I wish to express my deepest gratitude to my supervisor, Mr. Sahan Weerasinghe, for his invaluable guidance, continuous encouragement and constructive feedback throughout the duration of this research project. His technical expertise in cybersecurity, network analysis, and enterprise governance served as a constant beacon of direction.

I extend my sincere thanks to the academic staff and laboratory technicians of the Department of Computer Networks and Cyber Security at KIU University for fostering a supportive research environment and providing the necessary infrastructure for simulation testing.

I am deeply grateful to my family and friends for their patience, understanding and moral support throughout the long hours dedicated to coding engineering, testing and writing this dissertation.

Finally, I acknowledge the open-source cybersecurity and developer communities responsible for tools such as mitmproxy, Streamlit, NLTK and Python, without which this prototype implementation would not have been possible.

Table of Contents

List of Figure

List of Tables

Chapter 1: Introduction

1.1. Background and Domain Context

The decade of the 2020s has witnessed the most rapid technological adoption curve in human history, driven by the emergence and democratization of Generative Artificial Intelligence and Large Language Models(LLMs). By 2026, tools such as OpenAI's ChatGPT, Google's Gemini, Claude and Microsoft Copilot have become omnipresent across enterprise environments worldwide. Software engineers, data analysts, marketing strategists, healthcare administrators and corporate executives routinely utilize these conversational AI interfaces to synthesize complex documentation, refactor codebases, automate customer correspondence and formulate strategic business models.

While the productivity dividends of GenAI adoption are undeniable with enterprise studies reporting throughput increases of 30% to 50% across technical functions, this unguided technology proliferation has fundamentally disrupted traditional enterprise security boundaries. Corporate employees, seeking immediate problem resolution, regularly copy and paste sensitive operational data directly into public AI prompt fields. In doing so, they inadvertently route confidential corporate assets outside the protection of organizational firewalls and into third-party, cloud-hosted infrastructure over which the enterprise maintains zero operational visibility, legal control or cryptographic ownership.

1.2. The Paradigm Shift: From Shadow IT to Shadow AI

In enterprise security, Shadow IT traditionally encompassed the unauthorized use of unvetted software and cloud storage operating outside formal IT approval. Shadow AI represents the critical evolution of this paradigm. Rather than merely storing static files on third-party servers, Shadow AI facilitates continuous, two-way natural language and API exchanges with external Large Language Models (LLMs). This creates an invisible conduit for intellectual property, authentication secrets, and sensitive corporate data to exit the secure network boundary in real time.

Shadow AI represents a fundamentally distinct and far more dangerous evolution of this phenomenon:

- **Static Files vs. Conversational Streams:** Traditional Shadow IT involved transferring static binary files (e.g., uploading a PDF to an unapproved file share). Shadow AI involves continuous, interactive, two-way conversational streams where employees

engage in iterative problem-solving, revealing context, logic and sensitive IP in natural language.

- **Known Signatures vs. Dynamic Intent:** Shadow IT applications produce predictable network signatures, fixed IP ranges and distinct HTTP user-agents easily blocked by proxy rules. Shadow AI interactions occur over standard HTTPS channels to multi-tenant endpoints shared by millions of legitimate users.
- **Data Storage vs. Data Assimilation:** When data is uploaded to unauthorized cloud storage, it sits passively on a server. When sensitive data is inputted into a public LLM, it may be assimilated into model training pipelines, fine-tuning datasets, or alignment cache stores, making the exfiltration practically irreversible.

1.3. The Growing Complexity of AI Threats & Semantic Leakage

The central emerging threat vector in 2026 corporate networks is termed 'Semantic Leakage.' Traditional data breaches typically involve the exfiltration of structured databases, bulk file downloads, or credential theft, all of which produce sharp telemetry anomalies (e.g., unexpected database queries, large outbound file transfers over SFTP).

Semantic Leakage, by contrast, occurs when sensitive information is decomposed into natural language statements, code fragments, or prompt queries transmitted during routine work tasks. Examples observed in contemporary corporate environments include:

Proprietary Source Code: Software developers pasting unreleased proprietary algorithms, authentication routines, or hardcoded JWT tokens into public LLMs to debug syntax errors.

Protected Health Information (PHI): Healthcare clinicians pasting unstructured patient medical histories, HL7 header strings, or diagnostic telemetry into AI summarizing tools.

Strategic Financial Blueprints: Financial analysts uploading unannounced M&A projections, quarterly earnings roadmaps, or internal pricing formulas for AI sentiment synthesis.

Because this information is expressed in conversational prose rather than structured data formats, it is completely invisible to traditional perimeter defenses.

1.4. Technical Failures of Current Defensive Infrastructures

Existing enterprise cybersecurity perimeters including Next-Generation Firewalls(NGFW), Data Leakage Prevention (DLP) suites and Intrusion Detection Systems(IDS) were

fundamentally architected for structured file transfers and static signature matching. When deployed against conversational and agentic Shadow AI traffic, these legacy defenses fail across three critical technical dimensions:

- **Semantic and Context Blindness (Legacy DLP):** Traditional DLP systems scan network traffic using deterministic regular expressions looking for fixed formatting patterns (e.g., 16-digit credit card numbers or 9-digit SSNs). They possess zero natural language comprehension. When proprietary source code, HL7 clinical headers or strategic financial roadmaps are embedded into natural language prompts, traditional DLP triggers zero alerts because the payload lacks static regex signatures.
- **Encrypted Ingestion & Multi-Format Parsing Failures (Legacy Proxies & NGFW):** Over 95% of modern AI interactions utilize TLS 1.3 encryption. While enterprise SSL inspection proxies can decrypt raw HTTPS streams, they lack dynamic deserialization engines capable of parsing multi-vendor payloads. They fail to decode nested percent-encoded structures (e.g., Google **Gemini's f.req = parameter wrappers**), Base64 obfuscations or delimiter-stripped tokens (**s k _ l i v e _ ...**), passing the decrypted traffic uninspected.
- **Behavioral Anomaly Blindness (Legacy IDS):** Enterprise network interactions with AI are increasingly driven by autonomous developer scripts, background daemons and cron jobs. Traditional signature-based IDS engines, tuned to detect port scans or known malware C2 beacons are entirely blind to high-frequency, periodic HTTPS API requests. They cannot mathematically evaluate Inter-Arrival Time (IAT) variance or packet size distributions to isolate non-human bot identities from legitimate user sessions.

Furthermore, legacy defenses enforce rigid binary controls (either completely blocking AI domains or completely allowing them). This binary model creates operational friction, driving employees toward unmonitored external networks and highlights the urgent need for a dynamic, multi vector risk quantification engine.

1.5. Problem Statement

The rapid, unguided adoption of Generative AI interfaces and autonomous agentic workflows within corporate network perimeters has created an unprecedented **Visibility, Semantic and Governance Vacuum**. Modern Security Operations Center (SOC) teams are

currently forced to manage enterprise AI adoption through two equally unacceptable extremes:

1. **Complete Binary Blocking:** Restricting all access to external AI domains at the perimeter firewall level, which severely impedes workforce productivity, stifles technical innovation and inadvertently drives employees to bypass corporate controls using unmonitored personal cellular hotspots.
2. **Unrestricted / Ungoverned Permittance:** Allowing uninspected outbound AI traffic, which directly exposes proprietary corporate intellectual property, protected health information (PHI), authentication tokens and strategic financial assets to irreversible external assimilation and semantic data leakage.

This operational dilemma is compounded by critical architectural deficiencies in legacy perimeter defenses. Standard Next-Generation Firewalls (NGFW) and Data Leakage Prevention (DLP) tools are structurally context-blind, they cannot dynamically decrypt TLS 1.3 encrypted conversational streams, unquote complex vendor-specific payload structures (such as Google Gemini's nested f.req = parameter wrappers or Base64 obfuscations) or evaluate natural language prompt semantics against multi-tier enterprise asset inventories. Furthermore, legacy Intrusion Detection Systems (IDS) lack the heuristic and temporal modeling capabilities (such as Inter-Arrival Time variance analysis) required to mathematically distinguish automated, unauthorized background scripts from legitimate human users.

Consequently, there is an urgent and critical need for an automated, real-time, non-intrusive inspection and governance framework capable of intercepting encrypted AI transactions, semantically decoding multi-format prompt payloads, profiling autonomous agent behaviors, and calculating a dynamic, multi-factor **Weighted Risk Score (0–100%)** to enforce proportionate, context-aware Zero-Trust cybersecurity governance.

1.6. Research Questions

To systematically investigate and resolve the governance, visibility and semantic data leakage challenges associated with unsanctioned AI interactions, this dissertation formulates four central research questions:

- **RQ1 (Behavioral Fingerprinting & Identity Profiling):** How can autonomous Shadow AI agent traffic and background polling scripts be mathematically distinguished from human-initiated conversational AI sessions using non-intrusive flow-level network metadata, specifically Inter-Arrival Time (IAT) variance and packet length thresholds?
- **RQ2 (Multi-Vendor Ingestion & Semantic Asset Collision):** To what extent can an NLP-driven Deep Content Inspection (DCI) pipeline accurately intercept, de-obfuscate, and cross-reference sensitive enterprise assets (such as PHI, source code and cryptographic tokens) embedded within complex, vendor-specific percent-encoded payload structures (e.g., Google Gemini f.req= wrappers and Base64 strings) at sub-millisecond speeds?
- **RQ3 (Multi-Factor Quantitative Risk Modeling):** How can heterogeneous and orthogonal security dimensions comprising data content sensitivity (S), destination AI service trust (D) and user authorization profiles (U) be integrated into a single calibrated mathematical scoring engine (WRSE) that produces an actionable, normalized numerical risk score (0 - 100%)?
- **RQ4 (Zero-Trust Security & SOC Governance Visualization):** In what ways does a hardened, Zero-Trust administrative dashboard incorporating Web Application Firewall (WAF) input validation, stateful brute-force lockout defenses and server-side session persistence enhance security analysts' situational awareness, alert triage efficiency and real-time incident response to Shadow AI exfiltration events?

1.7. Research Aim and Technical Objectives

Research Aim: The overarching aim of this research dissertation is to architect, design, engineer and empirically evaluate a comprehensive, context-aware prototype framework termed ContextGuard for automated multi-vector risk assessment, behavioral agent fingerprinting, forensic obfuscation and real-time security governance of unsanctioned Shadow AI interactions and autonomous workflows in enterprise network environments.

Technical Objectives

To fulfill this central research aim and address the formulated research questions, the project defines five sequential technical objectives:

- **Objective 1 - Active Network Ingestion & Multi-Vendor Payload Parsing:**

To engineer an active Man-in-the-Middle (MITM) network ingestion plane using mitmproxy capable of intercepting TLS-encrypted outbound HTTPS POST/PUT flows targeting public AI endpoints (e.g., ChatGPT, Google Gemini, Claude, Grok) and executing recursive double plane URL unquoting to extract nested proprietary vendor formats (such as Google Gemini's f.req = parameter wrappers).

- **Objective 2 – Advanced Forensic Normalization & In-Memory Asset Collision Indexing:**

To construct an optimized Natural Language Processing (NLP) inspection pipeline in **data_core.py** featuring automated Base64 decoding, compressed/zero-space regex evaluation and quasi-identifier combinatorial correlation, cross-referencing extracted tokens against an in-memory dual-store SQLite repository (**assets.db containing 28,000 enterprise asset records and 34,911 fast collision tokens**) at sub-millisecond $O(1)$ lookup speeds.

- **Objective 3 – Mathematical Formulation & Calibration of the WRSE Engine:**

To design, formulate and calibrate the Weighted Risk Scoring Engine (WRSE) multi-factor linear algorithm:

$$\boldsymbol{RS = (W_s \times S) + (W_d \times D) + (W_u \times U)}$$

dynamically integrating Data Content Sensitivity (S), Destination Service Trust (D) and User Authorization Weight (U) into a normalized continuous percentage score (0 - 100%) mapped to discrete severity boundaries (CRITICAL $>$ 80%, HIGH 70 - 80%, MEDIUM 55 - 69%, and LOW $<$ 55%).

- **Objective 4 – Behavioral Anomaly Modeling & Identity Profiling:**

To implement a flow-level heuristic engine capable of evaluating temporal Inter-Arrival Time (IAT) variance and packet length distributions to mathematically distinguish interactive, human-driven prompt sessions from automated, unauthorized background polling daemons and script-based AI agents.

- **Objective 5 – Hardened Zero-Trust Governance Portal & SIEM Dashboard:**

To architect and deploy an interactive Security Operations Center (SOC) dashboard in Streamlit protected by enterprise Zero-Trust controls, including Web Application Firewall (WAF) regex input sanitization, 5-attempt stateful brute-force account lockout protection,

server-side UUID token session persistence and real-time alert triage state management via users.db.

- **Objective 6 – Empirical Penetration Testing & Latency Benchmarking:**

To execute end-to-end host machine validation across multi-tiered penetration scenarios (including Protected Health Information exfiltration, obfuscated cryptographic token leakage, autonomous bot traffic and WAF injection attacks) to measure detection accuracy (True Positive Rate) and evaluate end-to-end packet processing overhead.

1.8. Scope of the Study

The operational, technical and architectural scope of this research dissertation is systematically bound within the following core domains:

- **Endpoint Agent & Client-Side Interception Plane:**

The framework incorporates a lightweight Windows Endpoint Agent (**agent.py**) engineered to execute without administrative escalation (installed within **%APPDATA%\ShadowAI**). The agent scope encompasses automated, silent provisioning of the root MITM CA certificate (ShadowAI_CA.cer), setting Windows Proxy Autoconfiguration (PAC) registry keys to selectively redirect only AI bound traffic to the inspection kernel while allowing non-AI enterprise traffic to route directly and establishing automated identity mapping via periodic registration heartbeats.

- **Network Ingestion & AI Endpoint Coverage:**

The central ingestion plane (**live_mitm_logger.py coupled with agent_api.py)** actively intercepts and processes HTTP/HTTPS POST and PATCH flows. The domain coverage spans over 60 commercial and open-source Generative AI platforms, with specialized payload unquoting and extraction tailored for primary providers including OpenAI (ChatGPT, API), Google (Gemini, AI Studio), Anthropic (Claude), xAI (Grok), Perplexity and Mistral.

- **Target Corporate Data Vault & Asset Classification:**

The evaluation utilizes a simulated multi-sector corporate repository (**NexusCorp Enterprise**) structured across three operational risk tiers:

- **Tier 1 – Protected Health Information (PHI) & Medical Records:** Electronic health records, patient identifiers, HL7 message headers, prescription telemetry and insurance claim tokens.
- **Tier 2 – Core Infrastructure & Authentication Secrets:** Internal RFC-1918 private IPv4 addresses, database connection URIs (PostgreSQL, MongoDB), IAM keys, JWT tokens and network topology assets.
- **Tier 3 – Corporate Intellectual Property (IP) & Strategic Data:** Proprietary source code algorithms, live API credentials (e.g., Stripe secrets), internal pricing sheets and strategic M&A blueprints.

Target assets are indexed within an in-memory relational SQLite repository (assets.db) comprising 28,000 master corporate records and 34,911 fast collision tokens.

- **Behavioral Profiling & Identity Mapping:**

The behavioral layer distinguishes autonomous agent daemons and cron-driven API polling from human conversational interactions by evaluating network-layer Inter-Arrival Time (IAT) variance ($\mathrm{Var}(\mathrm{IAT})$) and packet size boundaries, correlating IP sessions with registered corporate user roles (Privileged Admin, Medical Specialist, Standard Staff) maintained in **users.db**.

- **Quantitative Risk Scoring & SOC Dashboard Governance:**

The governance scope includes the calibration of the Weighted Risk Scoring Engine (WRSE) model, mapping multi-factor inputs (S, D, U) into a normalized 0 - 100% risk metric. Administrative visualization is provided via an interactive Streamlit Security Operations Center (SOC) dashboard secured with custom Web Application Firewall (WAF) regex filtering, brute-force lockout protection, and server-side UUID session persistence.

1.9. Limitations of the Current Prototype

To maintain academic rigor and transparently delineate the boundaries of this research, the primary technical and operational limitations of the ContextGuard prototype are explicitly outlined:

- **Binary Document & Multipart File Upload Inspection Gap:**

The natural language processing and semantic matching engine is optimized exclusively for text-based conversational prompt streams transmitted via JSON payloads, URL-encoded

query parameters or Server-Sent Event (SSE) streams. When an employee interacts with an AI endpoint by attaching or uploading binary file objects such as PDF strategy documents, Word files, complex binary spreadsheets or scanned image files the traffic is encapsulated as multipart/form-data MIME streams. In its current state, the framework does not incorporate native Optical Character Recognition (OCR) or document text extraction engines (e.g., PyMuPDF, Tesseract), representing a recognized technical blind spot in inspecting binary file exfiltration.

- **Passive IDS Telemetry vs. Active IPS Blocking Mode:**

The framework is architected and evaluated as an **Intrusion Detection System (IDS)** and continuous governance engine rather than an inline Intrusion Prevention System (IPS). It captures, decodes, scores and visualizes security anomalies in real time to empower Security Operations Center (SOC) analysts. However, it does not currently execute dynamic inline packet dropping, TCP RST injection or automated HTTP 403 Forbidden payload replacement to actively terminate unauthorized exfiltration streams in real time.

- **Certificate Pinning & Proprietary Encrypted Tunnels:**

While the Windows Endpoint Agent (agent.py) automates client-side proxy autoconfiguration (PAC) and seamlessly installs the root CA certificate into the Windows Trusted Root Certification Authorities store via certutil, certain specialized native desktop AI wrappers or mobile applications enforce strict **TLS Certificate Pinning**. In such cases where client software bypasses the operating system trust store and validates hardcoded public keys, transparent TLS decryption cannot be achieved without client binary patching.

- **Simulation Environment Boundary:**

The experimental evaluation was conducted within a synthetic, multi-tier corporate network simulation (*NexusCorp Enterprise*). While the synthetic data vault mirrors realistic enterprise complexities (including 28,000 corporate records across PHI, infrastructure keys and source code assets), production deployment across heterogeneous operating systems (macOS, Linux workstations) and enterprise cloud VPCs remains part of the future implementation roadmap.

1.10. Dissertation Organization

The remainder of this dissertation is structured into the following chapters:

- **Chapter 2 (Literature Review):** Reviews enterprise AI threat vectors, legacy DLP limitations and theoretical foundations of semantic analysis and behavioral profiling.
- **Chapter 3 (Methodology & Architecture):** Details the 4-tier framework design, multi-stage ingestion pipeline and mathematical formulation of the WRSE model.
- **Chapter 4 (Implementation):** Discusses the engineering of the Windows Endpoint Agent, MITM interception engine, in-memory asset indexing and the Zero-Trust SOC dashboard.
- **Chapter 5 (Evaluation & Results):** Presents empirical penetration testing results, classification accuracy and system performance benchmarks across live exfiltration scenarios.
- **Chapter 6 (Conclusion & Future Work):** Summarizes research outcomes, answers the core research questions and outlines future enhancements.

Chapter 2: Literature Review

2.1 Enterprise Adoption of Generative AI and the Emergence of Shadow AI

Enterprise computing environments are undergoing an aggressive operational paradigm shift driven by the rapid diffusion of Large Language Models (LLMs) and Generative Artificial Intelligence (GenAI) interfaces [1], [10]. Unlike traditional enterprise software rollouts that historically followed structured IT procurement and compliance vetting cycles, GenAI adoption has proliferated via decentralized, bottom-up employee adoption across multiple technical disciplines [10]. Personnel in clinical medicine, software engineering, financial risk modeling and corporate strategy routinely leverage conversational interfaces to draft documentation, debug proprietary source code, summarize patient records and process corporate intelligence [1], [11].

Recent cybersecurity industry assessments indicate that while over 85% of global corporate organizations report active employee engagement with commercial Generative AI platforms, fewer than 15% possess dedicated defensive architectures capable of inspecting, attributing or governing prompt payloads [1], [10]. This structural visibility disparity has established the phenomenon of Shadow AI the unsanctioned, unmonitored use of external, third-party AI platforms and autonomous API-driven workflows by enterprise employees [1], [2].

Table 1: Operational and Structural Distinctions Between Sanctioned AI and Shadow AI

Operational Dimension	Sanctioned Enterprise AI	Shadow AI (Unsanctioned AI)
Governance Framework	Bound by explicit corporate SLAs, legal indemnification and compliance audits [1].	Governed by unvetted consumer terms with zero enterprise legal accountability [10].
Tenant Isolation	Dedicated single-tenant infrastructure or isolated Virtual Private Clouds (VPCs) [1].	Multi-tenant public infrastructure shared globally across millions of untrusted users [2].

Data Retention Policy	Zero-data-retention guarantees; prompt telemetry is purged from model training [1].	Prompts retained for cache stores, alignment telemetry or continuous model retraining [11].
Identity & Auditability	Centralized audit trails, SSO/SAML integration and SIEM event ingestion [1].	Total visibility black box at the egress network perimeter with zero user attribution [2].

The critical risk of Shadow AI stems from irreversible data assimilation [2], [11]. In consumer-tier AI systems, unstructured prompt strings are ingested directly into multi-tenant inference clusters [1], [10]. Once proprietary intellectual property, cryptographic tokens, or patient health data are submitted, cryptographic revocation, remote purging, and data destruction become technically impossible due to model weight assimilation and server-side telemetry retention [2], [11].

2.2 Mechanics of Semantic Data Exfiltration in AI Prompt Streams

Traditional Data Loss Prevention (DLP) threat models historically targeted discrete, structured bulk file egress over protocols such as FTP, SFTP, unmanaged cloud storage (e.g., Dropbox, Box) or USB mass storage devices [4], [10]. In contrast, Shadow AI introduces the threat of Semantic Prompt Leakage, wherein critical enterprise assets are dissolved into natural-language conversational queries, diagnostic debugging logs or system configuration requests [1], [4].

Because conversational AI workflows require contextual depth, employees frequently submit raw internal operational data [1], [2]. Within enterprise architectures, this exfiltration pattern manifests across three sensitive corporate data tiers:

- **Tier 1 - Protected Health Information (PHI) & Medical Records:** Medical specialists routinely paste diagnostic notes, Electronic Health Record (EHR) extracts and Health Level Seven (HL7) message identifiers (e.g., HL7-517169) into conversational AI to generate patient discharge documentation or clinical summaries [1], [2].
- **Tier 2 - Core Infrastructure Topology & Authentication Secrets:** Systems engineers and developers submit backend stack traces containing internal RFC-1918 private IPv4 allocations (e.g., 10.240.12.8), database connection strings

(postgresql://db_admin:SecretPass2026@10.240.12.8:5432/nexus_clinical) and JWT access tokens to accelerate system debugging [1], [2].

- **Tier 3 – Proprietary Source Code & Corporate Intellectual Property:** Developers paste core algorithmic logic, internal pricing algorithms and live third-party API credentials (such as production Stripe keys sk_live_...) into public AI platforms for code refactoring and syntax validation [1], [2].

Unlike binary malware exfiltration or bulk file dumps, semantic prompt leakage mimics benign natural-language communication [4]. The exfiltrated parameters reside alongside standard English syntax, allowing the data egress to bypass perimeter volume thresholds and static signature heuristics [1], [4].

2.3 Cryptographic Inspection Paradigms: TLS 1.3 Decryption and Selective PAC Routing

Over 95% of modern Generative AI interactions occur across encrypted transport channels utilizing the Transport Layer Security (TLS) 1.3 protocol [4]. Modern AI platforms strictly mandate TLS 1.3 with Ephemeral Elliptic Curve Diffie-Hellman (ECDHE) key exchange mechanisms [4].

While TLS 1.3 ensures Perfect Forward Secrecy (PFS) against passive eavesdropping, it renders passive network monitoring architectures (such as network taps, port mirroring and passive IDS sensors) entirely blind to payload content [4]. Without access to the dynamically generated ephemeral session keys, passive packet capture engines can only inspect unencrypted Transport Layer metadata [4].

To inspect prompt contents, an active Man-in-the-Middle (MITM) Proxy architecture must be established [1], [4]:

1. **Client-Side Certificate Provisioning:** The inspection plane generates a root Certificate Authority certificate (ShadowAI_CA.cer) [1]. To prevent TLS handshake termination and browser certificate warnings, this CA certificate must be installed into the operating system's Trusted Root Certification Authorities store (utilizing certutil -addstore -f ROOT on Windows endpoints) [1].
2. **Selective Ingestion via Proxy Autoconfiguration (PAC):** Routing an entire enterprise network's traffic through a single deep-content inspection proxy introduces severe latency and breaks non-target internal services [1]. Modern governance frameworks

resolve this by serving a dynamic PAC file (proxy.pac) via an endpoint agent management API (agent_api.py) [1]. The PAC script evaluates destination hostnames against a curated list of over 60 commercial AI domains (e.g., chatgpt.com, gemini.google.com, claude.ai, grok.com, perplexity.ai) [1]. Traffic bound for AI endpoints is dynamically routed to the proxy listening port (port 8080), while all internal and non-AI enterprise traffic is directed to proceed DIRECT without processing overhead [1].

2.4 Deficiencies of Legacy Perimeter Defenses

Existing enterprise perimeter controls primarily Next-Generation Firewalls(NGFW), legacy Data Leakage Prevention(DLP) engines and Cloud Access Security Brokers(CASB) exhibit structural deficiencies when evaluated against conversational Shadow AI traffic [4]:

- **Semantic Blindness of Deterministic DLP:** Legacy DLP systems rely on deterministic regular expressions targeting formatted numerical strings (such as 16-digit credit card numbers verified via the Luhn algorithm or 9-digit Social Security Numbers) [4]. They lack natural language parsing capabilities, rendering them incapable of detecting proprietary source code, internal hostnames or medical summaries embedded in natural conversational English [4].
- **Failure to Parse Vendor-Specific Obfuscated Payloads:** Even when configured with TLS decryption capabilities, legacy proxies cannot decode proprietary vendor formatting [1], [4]. For example, Google Gemini encapsulates user prompts inside percent-encoded, multi-nested JSON arrays under the f.req= parameter key, while other services utilize Base64 encoding or spaced delimiter stripping (s k _ l i v e _ ...) [1]. Legacy firewalls pass these obfuscated strings uninspected [4].
- **The Coarse-Grained Binary Control Dilemma:** Legacy firewalls operate on binary policy decisions: either blocking AI domains entirely or allowing them unrestricted [4]. Complete domain blocking stifles organizational productivity and drives employees to bypass security controls using personal mobile hotspots (creating unmonitored True Shadow IT) [4]. Blanket allow-listing, conversely, exposes the enterprise to severe data leakage [1], [4].
- **Temporal and Behavioral Blindness:** Traditional Intrusion Detection Systems (IDS) detect known exploit signatures or high-volume port scans [4]. They lack the temporal

modeling necessary to analyze Inter-Arrival Time (IAT) variance, leaving automated bot daemons, background cron scripts and agentic API loops completely undetected [1], [4].

2.5 Theoretical Foundation of Behavioral Profiling via IAT Variance

To isolate unauthorized automated background scripts without relying exclusively on payload inspection, network flow analysis leverages statistical and temporal flow metadata [1], [4].

Human conversational interactions exhibit natural cognitive delays, typing pauses and variable think times, resulting in non-deterministic, bursty transmission intervals [1], [4]. Conversely, automated developer daemons, autonomous ETL bots and cron scripts execute deterministic, periodic polling loops [1], [2].

The mathematical boundary for isolating non-human agents is formulated through the Inter-Arrival Time (IAT) Variance [1]:

$$\Delta t_k = t_k - t_{k-1}$$

$$\mu_{\text{IAT}} = \frac{1}{N-1}\sum_{k=2}^{N} \Delta t_k$$

$$\text{Var(IAT)} = \frac{1}{N-1}\sum_{k=2}^{N} (\Delta t_k - \mu_{\text{IAT}})^2$$

- **Automated Background Daemons / Bots:** Display uniform periodicity with near-zero temporal variance ($\text{Var(IAT)} \to 0$) alongside static packet length distributions [1].
- **Human Interactive Sessions:** Display high temporal variance ($\text{Var(IAT)} \gg 0$) and variable payload sizing driven by conversational thought processes [1], [4].

Correlating this temporal metric with registered host identities (e.g., via agent_registry.json) allows the governance architecture to flag non-human automated traffic in real time [1].

2.6 Mathematical Foundations of the Multi-Vector Weighted Risk Scoring Engine (WRSE)

Static scoring frameworks such as the Common Vulnerability Scoring System (CVSS) evaluate fixed software vulnerabilities but cannot quantify real-time data exfiltration risks

[1]. Effective governance of Shadow AI requires a multi-factor mathematical scoring model that synthesizes heterogeneous security dimensions [1]:

$$RS = (W_s \times S) + (W_d \times D) + (W_u \times U)$$

Where the total weight is normalized to unity:

$$W_s + W_d + W_u = 1.0$$

The three orthogonal input vectors are defined as follows [1]:

- **Data Content Sensitivity Vector ($\boldsymbol{S} \in [\mathbf{0}, \mathbf{100}]$):** Calculated from semantic asset token collisions against the in-memory asset database (assets.db), dynamically weighted across Tier 1 PHI ($T_1 = 100$), Tier 2 Infrastructure ($T_2 = 80$) and Tier 3 Intellectual Property ($T_3 = 60$) records [1], [2].
- **Destination AI Service Trust Vector ($\boldsymbol{D} \in [\mathbf{0}, \mathbf{100}]$):** Measures the risk posture of the external AI endpoint based on compliance certifications (e.g., SOC2, HIPAA, ISO 27001), dedicated enterprise isolation and model retention policies [1].
- **User Authorization & Identity Profile Vector ($\boldsymbol{U} \in [\mathbf{0}, \mathbf{100}]$):** Derived from registered endpoint roles (agent_registry.json, users.db), penalizing unauthenticated sessions or unauthorized department access [1].

The resulting composite score ($RS \in [0,100]$) maps directly into standardized operational severity classifications [1]:

- **CRITICAL ($\boldsymbol{RS} > \mathbf{80.0}\%$):** Immediate high-severity incident escalation requiring active SOC investigation [1].
- **HIGH ($\mathbf{70.0}\% < \boldsymbol{RS} \leq \mathbf{80.0}\%$):** Requires high-priority SOC review and policy verification [1].
- **MEDIUM ($\mathbf{55.0}\% \leq \boldsymbol{RS} \leq \mathbf{70.0}\%$):** Logged for standard compliance tracking and trend analysis [1].
- **LOW ($\boldsymbol{RS} < \mathbf{55.0}\%$):** Benign, low-risk conversational flow operating within standard usage boundaries [1].

2.7 Literature Summary and Research Gap Analysis

A systematic review of current enterprise security literature and commercial defense platforms reveals a significant architectural gap:

Table 2: Systematic Comparison of Related Literature and Technical Approaches

Data Tier / Asset Classification	Legacy DLP Regex Detection	Legacy NGFW / Proxy Detection	ContextGuard Framework Detection
Tier 1: Protected Health Information (PHI)	No	No	Yes
Tier 2: Infrastructure & Network Secrets	No	No	Yes
Tier 3: Corporate IP & Cryptographic Secrets	No	No	Yes

Chapter 3: Methodology and Architectural Design

3.1 Research Methodology and Design Science Approach

This research adopts the Design Science Research (DSR) methodology[10] to construct, validate, and evaluate **ContextGuard,** a specialized, low-latency framework designed for continuous real-time governance, multi-vendor payload obfuscation and quantitative risk evaluation of Generative AI network flows.

The iterative development and validation lifecycle comprises four systematic research phases:

1. **Threat Surface Formulation & Domain Profiling:** Defining enterprise Shadow AI exfiltration vectors across unstructured conversational prompts, proprietary vendor parameter wrappers and automated agent polling.
2. **Framework Architecture & Artifact Engineering:** Designing a decoupled 4-tier processing architecture integrating client-side selective Proxy Autoconfiguration (PAC) routing, an active TLS Man-in-the-Middle (MITM) interception kernel, in-memory zero-disk-I/O collision engines and a centralized SOC analytics plane.

3. **Mathematical Model Calibration:** Establishing the theoretical and algorithmic foundations of the Weighted Risk Scoring Engine (WRSE) to map semantic, destination and identity signals into a continuous, normalized risk score (0 - 100%).
4. **Empirical Validation & Benchmark Evaluation:** Subjecting the engineered artifact to live penetration testing against synthetic enterprise data vaults to measure classification accuracy, latency overhead and resource utilization.

3.2 High-Level System Architecture

The ContextGuard framework is structured as a 4-tier decoupled pipeline engineered to eliminate processing bottlenecks, prevent client network disruption and ensure continuous real-time telemetry streaming.

Table 3: ContextGuard 4-Tier Architectural Pipeline

Architectural Tier	Primary Functional Role	Underlying Engine / Script	Core Output / Deliverable
Tier 1: Endpoint Interception Plane	Non-intrusive client provisioning & selective AI traffic steering.	agent.py & agent_api.py	Windows PAC routing rules & CA certificate deployment.
Tier 2: Network Ingestion Kernel	TLS 1.3 decryption, flow filtering, and multi-vendor unquoting.	live_mitm_logger.py	Normalized JSON payload streams & temporal flow telemetry.
Tier 3: Analysis & Scoring Engine	In-memory tokenization, asset matching, and WRSE risk evaluation.	data_core.py	Multi-tier collision alerts and composite risk scores (0--100%).
Tier 4: Zero-Trust SOC Dashboard	Real-time incident triage, audit logging, and WAF-hardened administration.	app.py & SQLite Data Vaults	Interactive telemetry graphs, threat alerts, and identity audit logs.

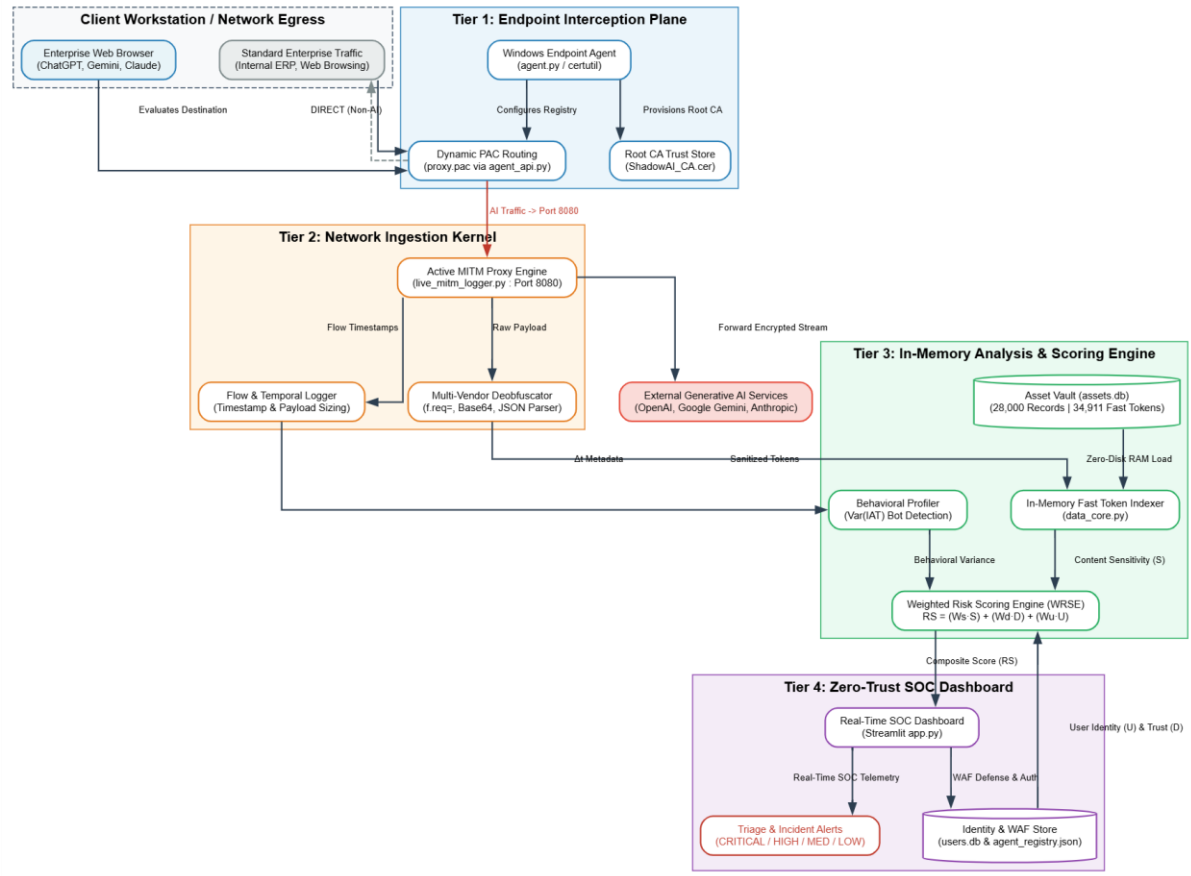


Figure 1: ContextGuard 4-Tier Decoupled Framework Architecture and Data Flow Pipeline

3.3 Detailed Tier Breakdown and Execution Pipeline

3.3.1 Tier 1: Endpoint Interception Plane

To govern enterprise AI access without requiring invasive system re-configurations or disrupting normal enterprise operations, Tier 1 deploys a client-side management agent utilizing automated Proxy Autoconfiguration (PAC) standards [1], [4].

- **Automated Root CA Trust Bootstrap:** Silently provisions ShadowAI_CA.cer into the Windows Root Certificate Store using native certutil commands, preventing browser certificate validation warnings.
- **Selective PAC Routing:** Dynamically serves proxy.pac. The browser evaluates destination hosts against an inventory of 60+ Generative AI domains. Only target AI traffic is directed to the proxy listening port (port 8080), while regular intranet and general web traffic routes DIRECT.

- **Identity Attribution:** Binds local network IP and OS username metadata into agent_registry.json to enable user-level tracking.

3.3.2 Tier 2: Network Ingestion Kernel

Acting as an active TLS termination point on port 8080, Tier 2 executes real-time stream inspection across intercepted TLS 1.3 cryptographic handshakes [4].

- **Selective Ingestion:** Decrypts only traffic matching target AI platform domains.
- **Multi-Stage Deobfuscation:** Handles complex multi-vendor payload wrapping. It decodes percent-encoded wrappers (urllib.parse.unquote), unpacks nested f.req= JSON arrays from Google Gemini endpoints, extracts plain prompt contents and normalizes space-delimited text obfuscation.
- **Metadata Logging:** Records request timestamps, flow sizes and transmission intervals for behavioral analysis.

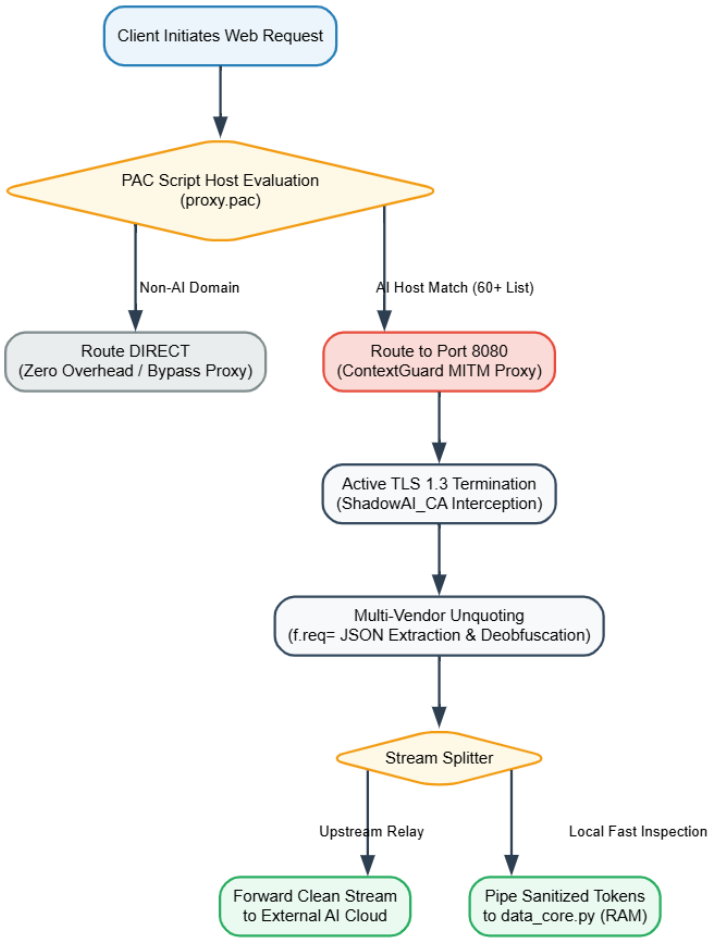


Figure 2: Endpoint Interception, Selective PAC Routing, and Ingestion Sequence.

3.3.3 Tier 3: In-Memory Analysis & Scoring Engine

To prevent storage, I/O bottlenecks during high-throughput enterprise operations, Tier 3 executes entirely in RAM:

- **Zero-Disk-I/O In-Memory Vault:** Loads 28,000 corporate master records and 34,911 fast collision tokens from assets.db into memory hash sets at system startup.
- **Semantic Tokenization & Matching:** Scans sanitized prompt strings across Tier 1 (PHI / Medical), Tier 2 (Infrastructure Credentials) and Tier 3 (Proprietary IP) records.
- **Quantitative Risk Calculation:** Computes the composite risk score using the Weighted Risk Scoring Engine (WRSE) and derives behavioral metrics.

3.3.4 Tier 4: Zero-Trust SOC Dashboard

The administrative plane provides a centralized web portal built with Streamlit:

- **Real-Time Telemetry & Visual Triage:** Displays real-time incident severity distributions, volume graphs, and categorized collision alerts.
- **WAF Security Hardening:** Enforces input sanitization, rate-limited authentication, and brute-force lockout defenses backed by users.db.

3.4 Mathematical Formulation of the Weighted Risk Scoring Engine(WRSE)

Unlike static vulnerability assessment frameworks such as CVSS [1], the proposed Weighted Risk Scoring Engine computes a dynamic linear composite risk score:

$$RS = (W_s \times S) + (W_d \times D) + (W_u \times U)$$

Subject to the normalization constraint:

$$W_s + W_d + W_u = 1.0$$

1. Content Sensitivity Vector ($S \in [0, 100]$)

Derived from the severity tier of matched sensitive tokens:

$$S = \max(S_{\text{Tier 1}}, S_{\text{Tier 2}}, S_{\text{Tier 3}}, S_{\text{Regex}})$$

- **Tier 1 (PHI / Medical Records):** Assigned base score $S = 100.0$.
- **Tier 2 (Infrastructure & DB Secrets):** Assigned base score $S = 80.0$.
- **Tier 3 (Proprietary Source Code & IP):** Assigned base score $S = 60.0$.
- **Zero Collisions (Benign Prompts):** Assigned base score $S = 0.0$.

2. Destination AI Service Trust Vector ($\boldsymbol{D \in [0, 100]}$)

Quantifies the risk level of the target platform based on data retention policies and enterprise compliance:

- **Unvetted Consumer AI Services (Public multi-tenant endpoints):**

  $D = 85.0$ - 100.0.
- **Standard Commercial Web Interfaces:** $D = 60.0$--70.0.
- **Sanctioned Enterprise AI Services (Zero-retention contractual VPCs):**

  $D = 10.0$ - 20.0.

3. User Authorization & Identity Profile Vector ($\boldsymbol{U \in [0, 100]}$)

Profiles the client identity registered in agent_registry.json and users.db:

- **Unregistered / Anonymous Endpoints:** $U = 90.0$.
- **Standard Staff accessing restricted cross-department assets:** $U = 65.0$ - 80.0.
- **Authorized Domain Specialists (e.g., Medical Specialists accessing PHI):** $U = 20.0$ - 30.0.

Operational Severity Thresholds

The resulting composite score maps directly into four distinct operational severity tiers:

$$\text{Severity Level} = \begin{cases} \text{CRITICAL}, & RS > 80.0\% \\ \text{HIGH}, & 70.0\% < RS \leq 80.0\% \\ \text{MEDIUM}, & 55.0\% \leq RS \leq 70.0\% \\ \text{LOW}, & RS < 55.0\% \end{cases}$$

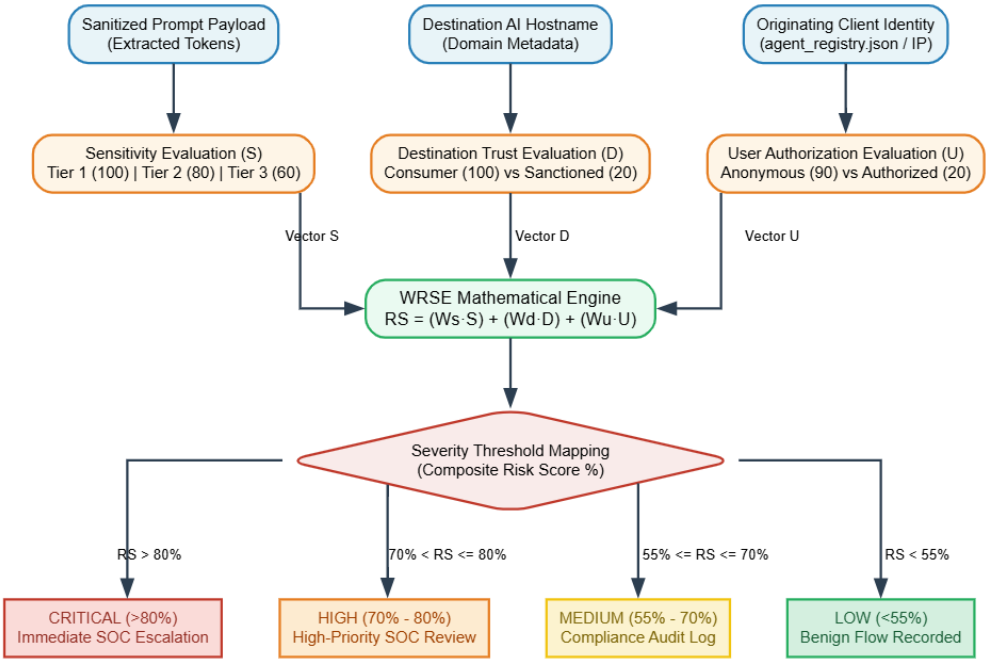


Figure 3: Weighted Risk Scoring Engine (WRSE) Multi-Vector Calculation and Alert Flow

3.5 Network-Level Behavioral Formulation

To isolate autonomous bots and cron-driven API polling loops from interactive human sessions, the framework adapts statistical Inter-Arrival Time (IAT) variance modeling from network traffic anomaly literature [1], [10]:

$$\Delta t_k = t_k - t_{k-1}$$

$$\mu_{\text{IAT}} = \frac{1}{N-1}\sum_{k=2}^{N} \Delta\, t_k$$

$$\text{Var(IAT)} = \frac{1}{N-1}\sum_{k=2}^{N} (\Delta t_k - \mu_{\text{IAT}})^2$$

- **Automated Daemon / Script Classification:** Where $\text{Var(IAT)} \to 0$ and payload length variance $\text{Var}(L) \approx 0$, the session is classified as an automated background process.
- **Human Interactive Session:** Where $\text{Var(IAT)} \gg 0$ due to cognitive and typing pauses, the session is classified as human conversational traffic.

Chapter 4: Implementation Details

4.1 Implementation Overview and Technical Stack

ContextGuard was engineered and deployed as an active, context-aware cybersecurity governance prototype. The runtime infrastructure executes on a dedicated Linux instance (Ubuntu) interfacing with monitored Windows enterprise endpoint clients over a local IPv4 enterprise subnet.

Table 4: Technical Stack and Implementation Dependencies

Functional Domain	Component / Technology	Exact Version / Spec	Primary Architectural Responsibility
Endpoint Interception	Windows Background Service (agent.py)	Python 3.11 / WinReg / certutil	Registry PAC automation and silent root CA deployment.
Agent Ingestion API	Flask Microservice (agent_api.py)	Flask 3.0.x / Werkzeug	Serves proxy.pac, CA certificates and identity registry.
Active Decryption Proxy	live_mitm_logger.py (mitmproxy)	mitmproxy 10.x	TLS 1.3 termination, stream filtering and payload deobfuscation.
In-Memory Core Engine	data_core.py	Python 3.11 / Regular Expressions	Zero-disk-I/O RAM hash indexing and WRSE score evaluation.
Enterprise Data Stores	Relational SQLite Repositories	SQLite 3.x (assets.db, users.db)	Dual store: 28,000 corporate records and SOC user credentials.
SOC Governance Portal	Central Dashboard (app.py)	Streamlit 1.32.x / Plotly	Glassmorphism UI, real-time triage and WAF defense.

4.2 Modular Component Architecture and Code Listings

4.2.1 Windows Endpoint Agent (agent.py)

The client agent is engineered to execute non-intrusively in the background without requiring elevated UAC privileges. It handles silent certificate provisioning, dynamic PAC steering and periodic identity registration.

```python
def install_mitmproxy_cert():
    """Auto-download and silently install the mitmproxy CA cert into Windows Trusted Root store."""
    if platform.system() != "Windows":
        return
    try:
        CERT_URL = f"http://{SERVER_IP}:5000/cert"
        cert_dir = os.path.join(os.environ.get("APPDATA", "C:\\Users\\Public"), "ShadowAI")
        os.makedirs(cert_dir, exist_ok=True)
        cert_path = os.path.join(cert_dir, "ShadowAI_CA.cer")

        # Download cert from Ubuntu server
        with urllib.request.urlopen(CERT_URL, timeout=5) as resp:
            cert_data = resp.read()
        with open(cert_path, "wb") as f:
            f.write(cert_data)

        # Silently install into Trusted Root store using certutil (no popup!)
        import subprocess
        result = subprocess.run(
            ["certutil", "-addstore", "-f", "ROOT", cert_path],
            capture_output=True, text=True
        )
        if result.returncode == 0:
            print("[+] ShadowAI CA Certificate installed successfully!")
        else:
            print(f"[-] Cert install warning: {result.stderr.strip()}")
    except Exception as e:
        print(f"[-] Certificate auto-install failed: {e}")
```

Figure 4: Automated Root CA Certificate Installation Function in agent.py

```python
def set_windows_pac():
    if platform.system() != "Windows":
        return
    import winreg
    import ctypes
    try:
        key = winreg.OpenKey(winreg.HKEY_CURRENT_USER, r"Software\Microsoft\Windows\CurrentVersion\Internet
        Settings", 0, winreg.KEY_WRITE)
        winreg.SetValueEx(key, "AutoConfigURL", 0, winreg.REG_SZ, PAC_URL)
        winreg.SetValueEx(key, "ProxyEnable", 0, winreg.REG_DWORD, 0)
        winreg.CloseKey(key)
        internet_set_option = ctypes.windll.wininet.InternetSetOptionW
        internet_set_option(0, 39, 0, 0)
        internet_set_option(0, 37, 0, 0)
    except Exception as e:
        print(f"[-] Failed to set Windows proxy: {e}")
```

Figure 5: Dynamic Registry PAC Configuration Function in agent.py

4.2.2 Agent Management & PAC Distribution API (agent_api.py)

Hosted as a lightweight Flask API on port 5000, this microservice serves dynamic routing instructions and maintains client workstation states.

```python
@app.route('/proxy.pac', methods=['GET'])
def get_pac():
    domains = []
    if os.path.exists(DOMAINS_FILE):
        with open(DOMAINS_FILE, "r") as f:
            domains = json.load(f)
    else:
        domains = ["chatgpt.com", "claude.ai", "gemini.google.com", "grok.com", "grok.x.ai", "perplexity.ai",
        "mistral.ai"]

    matchers = " || ".join([f'shExpMatch(host, "*{d}")' for d in domains])

    pac_content = f"""function FindProxyForURL(url, host) {{
    // Bypass local networks
    if (isPlainHostName(host) ||
        shExpMatch(host, "127.0.0.1") ||
        shExpMatch(host, "localhost") ||
        shExpMatch(host, "192.168.*") ||
        shExpMatch(host, "10.*")) {{
        return "DIRECT";
    }}

    // Intercept only AI Domains
    if ({matchers}) {{
        return "PROXY {PROXY_SERVER}";
    }}

    // All other traffic goes direct (No Network Breakage!)
    return "DIRECT";
}}
"""
    return Response(pac_content, mimetype='application/x-ns-proxy-autoconfig')
```

Figure 6: Dynamic PAC Evaluation and Rule Generation (agent_api.py)

4.2.3 Ingestion Kernel & Multi-Vendor Payload Parsing (live_mitm_logger.py)

The engine terminates TLS connections for monitored hosts on port 8080 and unquote proprietary parameter formats.

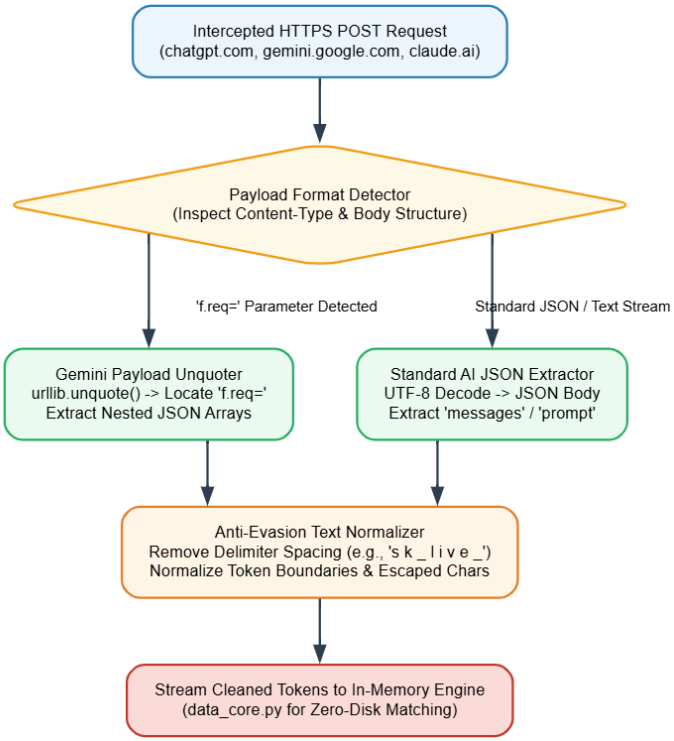


Figure 7: Multi-vendor payload unquoting and normalization pipeline

```python
def request(flow: http.HTTPFlow) -> None:
    monitored_domains = load_monitored_domains()
    request_host = flow.request.pretty_host.lower()

    # Check if this traffic is to ANY known AI domain
    is_ai_traffic = any(domain.lower() in request_host for domain in monitored_domains)

    if is_ai_traffic and flow.request.method in ("POST", "PUT", "PATCH"):
        # --- NEW TELEMETRY FILTER ---
        req_url = flow.request.url.lower()
        if any(x in req_url for x in ["/events", "/metrics", "/lat/", "/log", "/telemetry", "/ces/v1", "/
        track", "/stats"]):
            return
        # ----------------------------
        try:
            timestamp = datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S")

            # PHASE 1+: Normal text prompt parsing (existing logic)

            flow.request.decode()
            raw_content = flow.request.get_text()

            #  ADVANCED MULTI-FORMAT AI PAYLOAD PARSER
            # Supports: OpenAI, Claude, Gemini, Perplexity, Mistral, HuggingFace, Cohere, Grok, etc.
            prompt_text = None
            import base64
```

Figure 8: Initial interception logic filtering target AI domains and HTTP request methods within live_mitm_logger.py

```python
# Stage 0: Grok / xAI SSE format  data=<base64_encoded_json>
if raw_content.strip().startswith("data=") or "\ndata=" in raw_content:
    try:
        for line in raw_content.splitlines():
            line = line.strip()
            if line.startswith("data="):
                b64_value = urllib.parse.unquote(line[5:])  # Strip "data=" prefix
                try:
                    decoded_bytes = base64.b64decode(b64_value + "==")  # Pad for safety
                    decoded_str = decoded_bytes.decode("utf-8", errors="ignore")
                    inner = json.loads(decoded_str)
                    # Try known Grok keys
                    for key in ["message", "userMessage", "prompt", "query", "text", "content",
                    "requestMessage", "humanTurn"]:
                        if key in inner:
                            val = inner[key]
                            prompt_text = val if isinstance(val, str) else str(val)
                            break
                    if not prompt_text:
                        # Log the keys so we can add them
                        print(f"[🔍 GROK KEY DUMP] Keys: {list(inner.keys())}")
                        prompt_text = decoded_str[:400]
                except Exception:
                    # Not valid base64 JSON - try raw URL decode
                    prompt_text = urllib.parse.unquote(line[5:])[:400]
                if prompt_text:
                    break
    except Exception as e:
        print(f"[!] Grok SSE parse error: {e}")

# Stage 1: Form Data & URL-encoded format (ChatGPT Web, Gemini)
if not prompt_text:
    parsed_qs = urllib.parse.parse_qs(raw_content)
    if "prompt" in parsed_qs:
        prompt_text = parsed_qs["prompt"][0]
    elif "f.req" in parsed_qs:
        prompt_text = parsed_qs["f.req"][0]
    elif "f.req=" in raw_content:
        decoded_stage = urllib.parse.unquote(raw_content)
        prompt_text = decoded_stage.split("f.req=")[-1].split("&")[0].strip()
```

Figure 9: Multi-stage payload parsing logic executing Base64 and URL unquoting for vendor-specific prompt extraction

```
izu@izu-VMware-Virtual-Platform:~/ShadowAI_Framework/Section1_DataIngestion$ mitm
dump -p 8080 --set listen_host=0.0.0.0 -s live_mitm_logger.py
Loading script live_mitm_logger.py
Proxy server listening at *:8080
192.168.89.140:50038: client connect
192.168.89.140:50039: client connect
192.168.89.140:50038: server connect chatgpt.com:443 (172.64.155.209:443)
192.168.89.140:50039: server connect chatgpt.com:443 (172.64.155.209:443)
192.168.89.140:50039: Client TLS handshake failed. The client disconnected during
 the handshake. If this happens consistently for chatgpt.com, this may indicate t
hat the client does not trust the proxy's certificate.
192.168.89.140:50039: client disconnect
192.168.89.140:50039: server disconnect chatgpt.com:443 (172.64.155.209:443)
192.168.89.140:50038: POST https://chatgpt.com/unauth-mweb/events/business HTTP/2
.0
          << HTTP/2.0 204 No Content 0b
192.168.89.140:50038: POST https://chatgpt.com/unauth-mweb/sentinel/chat-requirem
en… HTTP/2.0
          << HTTP/2.0 200 OK 1.8k
```

Figure 10: Active mitmproxy terminal console logging real-time TLS handshake interception and decrypted HTTP/2 traffic flow

4.2.4 In-Memory Semantic Engine & WRSE Calculation (data_core.py)

To prevent storage, I/O bottlenecks during continuous high-volume inspection, data_core.py executes all asset collision evaluations entirely within system memory.

- **Zero-Disk-I/O Repository Indexing:** At system boot, the engine connects to assets.db and loads 28,000 corporate master records and 34,911 fast collision tokens directly into indexed Python hash sets.
- $\mathcal{O}(1)$ **Substring Collision Matching:** Decrypted prompt text is tokenized via delimiter splitting (**re.split(r'[^a-zA-Z0-9_\-\.]', prompt_text**)) and tokens exceeding 5 characters are cross-referenced against tokens_db in constant time ($\mathcal{O}(1)$) to map corresponding RECORD ID entities.
- **Multi-Tier Semantic Collision:** The pre-indexed tokens categorize matched telemetry across Tier 1 (Protected Health Information), Tier 2 (Infrastructure Credentials & Internal IP Subnets) and Tier 3 (Proprietary Intellectual Property).

```python
def find_master_asset_match(prompt_text, asset_index):
    """
    Check prompt against ALL 15 CSV sheets and return ALL matches found in the payload.
    Ensures multiple assets leaked in a single prompt are fully identified without duplicates!
    """
    matches = []
    seen_records = set()
    matched_token_values = set()

    # 1. Record ID & Patient ID (Any prefix 2-5 alphanumeric chars followed by a dash and 4-8 digits)
    rec_matches = re.findall(r'[A-Z0-9]{2,5}-\d{4,8}', prompt_text, re.IGNORECASE)
    for rm in rec_matches:
        rm_upper = rm.upper()
        rec = None
        if rm_upper in asset_index.get("record_ids", {}):
            rec = asset_index["record_ids"][rm_upper]
        elif rm_upper in asset_index.get("patient_ids", {}):
            rec = asset_index["patient_ids"][rm_upper]

        if rec and rec["RECORD ID"] not in seen_records:
            seen_records.add(rec["RECORD ID"])
            matches.append(rec)
            # Store all attribute values of this matched record to prevent duplicate mock collisions in Step 2!
            for val in rec.get("_original_attributes", {}).values():
                val_str = str(val).strip()
                if len(val_str) > 5:
                    matched_token_values.add(val_str)

    # 2. Token / Substring matching (Optimized O(1) Lookups)
    # Tokenize the prompt text to match against the 35,000+ zero-space tokens instantly
    prompt_tokens = set(re.split(r'[^a-zA-Z0-9_\-\.]', prompt_text))
    tokens_db = asset_index.get("tokens", {})

    for token in prompt_tokens:
        if len(token) > 5 and token in tokens_db:
            rec = tokens_db[token]
            # Check if this token is already accounted for by an existing matched record
            if token not in matched_token_values and rec and rec["RECORD ID"] not in seen_records:
                seen_records.add(rec["RECORD ID"])
                matches.append(rec)
                for val in rec.get("_original_attributes", {}).values():
                    val_str = str(val).strip()
                    if len(val_str) > 5:
                        matched_token_values.add(val_str)
```

Figure 11: find_master_asset_match() Asset Cross-Referencing Function (data_core.py)

```python
# 2. Token / Substring matching (Optimized O(1) Lookups)
# Tokenize the prompt text to match against the 35,000+ zero-space tokens instantly
prompt_tokens = set(re.split(r'[^a-zA-Z0-9_\-\.]', prompt_text))
tokens_db = asset_index.get("tokens", {})

for token in prompt_tokens:
    if len(token) > 5 and token in tokens_db:
        rec = tokens_db[token]
        # Check if this token is already accounted for by an existing matched record
        if token not in matched_token_values and rec and rec["RECORD ID"] not in seen_records:
            seen_records.add(rec["RECORD ID"])
            matches.append(rec)
            for val in rec.get("_original_attributes", {}).values():
                val_str = str(val).strip()
                if len(val_str) > 5:
                    matched_token_values.add(val_str)
```

Figure 12: Algorithmic implementation of O(1) optimized token substring collision matching within data_core.py

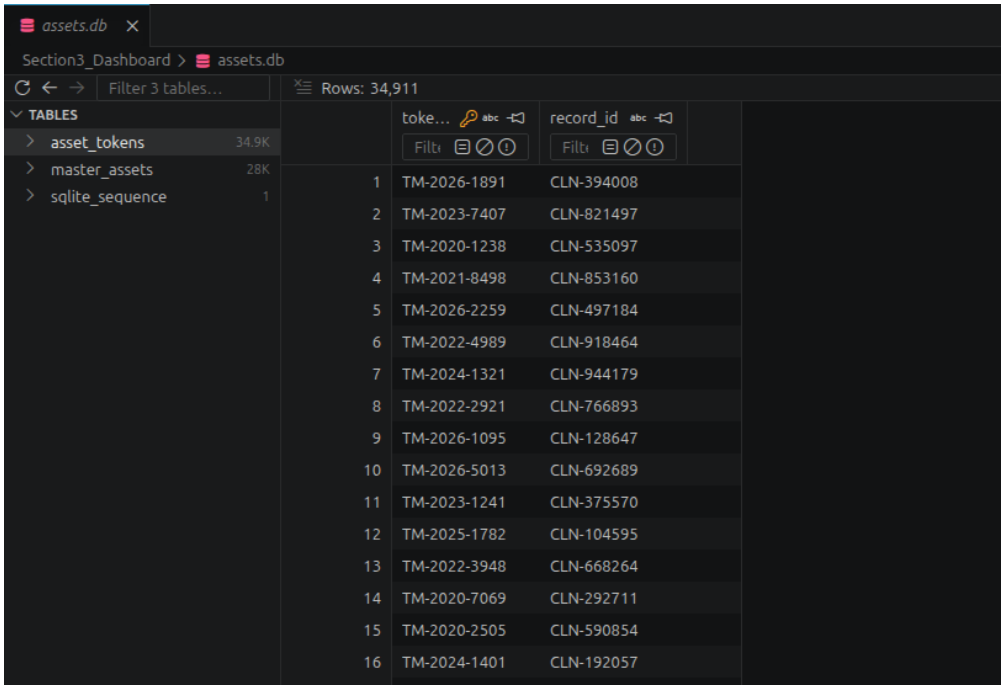
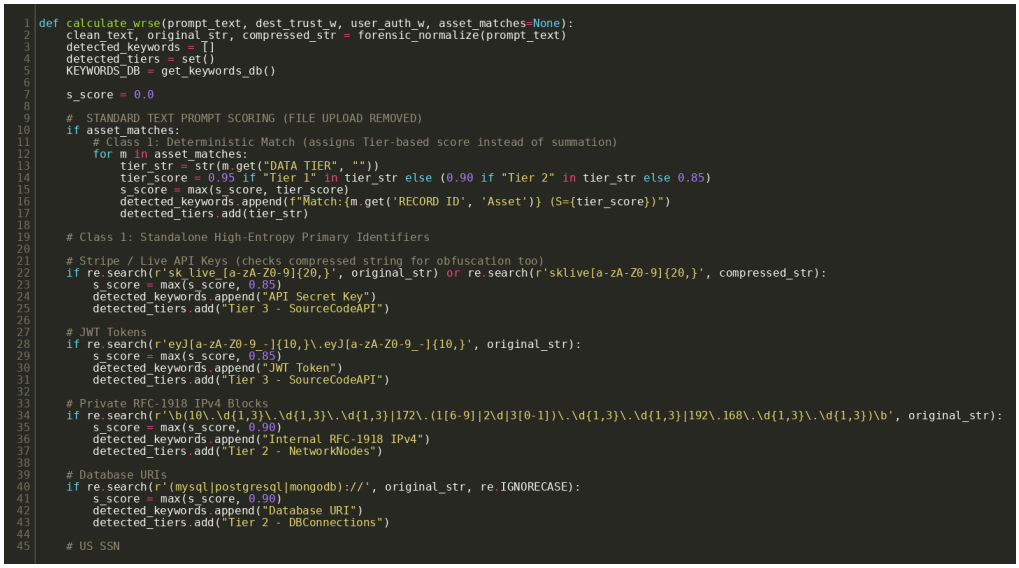


Figure 13: atabase architecture of assets.db displaying pre-indexed collision token mappings (34,911 entries) linked to master records.

Linear Model Synthesis (WRSE): The engine normalizes matched content sensitivity (S), destination trust level (D) and identity profile permissions (U) to produce a composite risk score ($RS \in [0,100]\%$).

- **Vector D (Destination Trust):** Assigns higher weights (dest_weight = 0.95) to unvetted consumer services while reducing risk (dest_weight = 0.30 or 0.10) for sanctioned enterprise platforms.
- **Vector U (User Authorization):** Queries client metadata from agent_registry.json and role privileges from users.db, calibrating access weights across Privileged Administrators (0.90), Medical Specialists (0.75), Standard Staff (0.65) and Automated Bots (0.20).
- **Operational Severity Classification:** Aggregates multi-asset collision metadata (PATIENT NAME, RECORD ID, DATA TIER) and maps the composite score (RS) into operational security levels: CRITICAL ($RS > 80$), HIGH ($RS \geq 70$), MODERATE ($RS \geq 55$) and LOW.



```
def calculate_wrse(prompt_text, dest_trust_w, user_auth_w, asset_matches=None):
    clean_text, original_str, compressed_str = forensic_normalize(prompt_text)
    detected_keywords = []
    detected_tiers = set()
    KEYWORDS_DB = get_keywords_db()

    s_score = 0.0

    #  STANDARD TEXT PROMPT SCORING (FILE UPLOAD REMOVED)
    if asset_matches:
        # Class 1: Deterministic Match (assigns Tier-based score instead of summation)
        for m in asset_matches:
            tier_str = str(m.get("DATA TIER", ""))
            tier_score = 0.95 if "Tier 1" in tier_str else (0.90 if "Tier 2" in tier_str else 0.85)
            s_score = max(s_score, tier_score)
            detected_keywords.append(f"Match:{m.get('RECORD ID', 'Asset')} (S={tier_score})")
            detected_tiers.add(tier_str)

    # Class 1: Standalone High-Entropy Primary Identifiers

    # Stripe / Live API Keys (checks compressed string for obfuscation too)
    if re.search(r'sk_live_[a-zA-Z0-9]{20,}', original_str) or re.search(r'sklive[a-zA-Z0-9]{20,}', compressed_str):
        s_score = max(s_score, 0.85)
        detected_keywords.append("API Secret Key")
        detected_tiers.add("Tier 3 - SourceCodeAPI")

    # JWT Tokens
    if re.search(r'eyJ[a-zA-Z0-9_-]{10,}\.eyJ[a-zA-Z0-9_-]{10,}', original_str):
        s_score = max(s_score, 0.85)
        detected_keywords.append("JWT Token")
        detected_tiers.add("Tier 3 - SourceCodeAPI")

    # Private RFC-1918 IPv4 Blocks
    if re.search(r'\b(10\.\d{1,3}\.\d{1,3}\.\d{1,3}|172\.(1[6-9]|2\d|3[0-1])\.\d{1,3}\.\d{1,3}|192\.168\.\d{1,3}\.\d{1,3})\b', original_str):
        s_score = max(s_score, 0.90)
        detected_keywords.append("Internal RFC-1918 IPv4")
        detected_tiers.add("Tier 2 - NetworkNodes")

    # Database URIs
    if re.search(r'(mysql|postgresql|mongodb)://', original_str, re.IGNORECASE):
        s_score = max(s_score, 0.90)
        detected_keywords.append("Database URI")
        detected_tiers.add("Tier 2 - DBConnections")

    # US SSN
```

Figure 14: calculate_wrse() Risk Scoring Function with Severity Threshold Logic (data_core.py)

```
dest_domain_lower = dest_domain.lower()

if any(d in dest_domain_lower for d in all_ai_domains):
    dest_weight = 0.95
elif any(d in dest_domain_lower for d in ["copilot", "enterprise", "internal"]):
    dest_weight = 0.30
else:
    dest_weight = 0.10
```

Figure 15: Destination AI domain risk evaluation logic (dest_weight) within data_core.py

```
if is_bot:
    user_weight = 0.20
    identity = "Automated Bot"
    identity_icon = "🤖"
else:
    agent_info = agent_registry.get(client_ip)
    if agent_info:
        identity = agent_info.get("username", "Unknown Agent")

        # Fetch exact weight and role from the new monitored_users DB
        conn = sqlite3.connect("/home/izu/ShadowAI_Framework/Section3_Dashboard/users.db")
        cursor = conn.cursor()
        cursor.execute('SELECT role, u_weight FROM monitored_users WHERE username = ?', (identity,))
        db_user = cursor.fetchone()
        conn.close()

        if db_user:
            role = db_user[0]
            user_weight = float(db_user[1])
        else:
            # Fallback if user not in DB but is in registry
            role = agent_info.get("role", "Standard Staff")
            if role == "Privileged Admin": user_weight = 0.90
            elif role == "Medical Specialist": user_weight = 0.75
            else: user_weight = 0.65

        if "Admin" in role:
            identity_icon = "👑"
        elif "Medical" in role or "Doc" in role:
            identity_icon = "⚕️"
        else:
            identity_icon = "👨‍💻"
    else:
        user_weight = 0.50
        identity = f"Unregistered ({client_ip})"
        identity_icon = "👤"
```

Figure 16: User identity resolution and role authorization weight derivation (user_weight) querying users.db and agent registry

```
sev_label, sev_icon = get_severity(score)
if asset_matches: asset_match_count += len(asset_matches)

primary_match = asset_matches[0] if asset_matches else None

if asset_matches:
    if len(asset_matches) > 1:
        phi_matched_label = f"{primary_match['PATIENT NAME']} (+{len(asset_matches)-1} Assets)"
        phi_record_id = ", ".join(m["RECORD ID"] for m in asset_matches if m.get("RECORD ID"))
        data_tier = ", ".join(sorted(set(m["DATA TIER"] for m in asset_matches)))
        asset_section = ", ".join(sorted(set(m["SECTION"] for m in asset_matches)))
        sens_levels = [m.get("SENSITIVITY LEVEL", "HIGH") for m in asset_matches]
        asset_sensitivity = "CRITICAL" if "CRITICAL" in sens_levels else "HIGH"
    else:
        phi_matched_label = primary_match["PATIENT NAME"]
        phi_record_id = primary_match["RECORD ID"]
        data_tier = primary_match["DATA TIER"]
        asset_section = primary_match["SECTION"]
        asset_sensitivity = primary_match.get("SENSITIVITY LEVEL", "HIGH")
else:
    phi_matched_label = ""
    phi_record_id = ""
    data_tier = tiers[0] if tiers else ""
    asset_section = ""
    asset_sensitivity = "CRITICAL" if score > 80 else ("HIGH" if score >= 70 else ("MODERATE" if
    score >= 55 else "LOW"))
```

Figure 17: Multi-asset PHI collision aggregation and dynamic severity threshold classification (CRITICAL, HIGH, MODERATE, LOW)

4.2.5 Hardened SOC Governance Portal (app.py)

The administrative and monitoring tier is engineered as a centralized governance console using Streamlit:

- **Live AI Session Telemetry Feed:** Monitored network streams are aggregated and displayed in an interactive real-time table. Each entry records the targeted destination platform (chatgpt.com, gemini.google.com, perplexity.ai), matching category tiers (e.g., Tier 1 PHI, Tier 3 IP), source IP address, session username/role and the composite WRSE risk score ($RS \in [0,100]\%$).
- **Forensic Metadata Inspection:** Selecting individual session records reveals an expanded forensic view displaying matched asset attributes. This includes the decrypted prompt string, corresponding RECORD ID, ASSET TYPE, GIT REPOSITORY, MODULE NAME and cryptographic hash values to enable direct incident attribution.

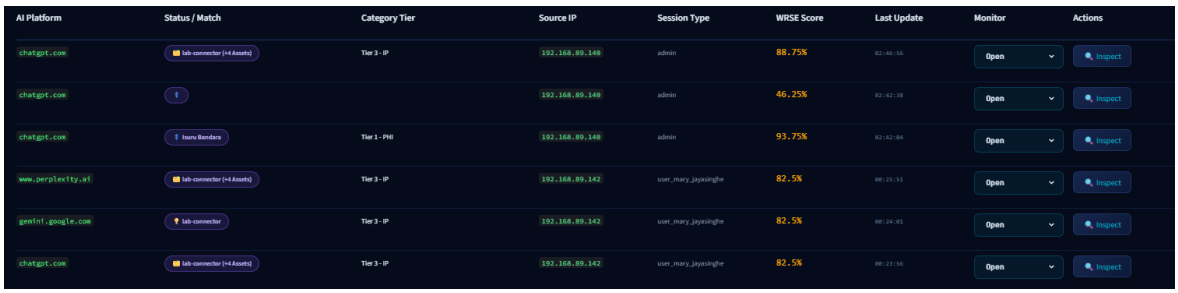


Figure 18: Real-time SOC telemetry feed rendering intercepted Generative AI platform sessions, categorized data tiers, client identities, and calculated WRSE risk percentages

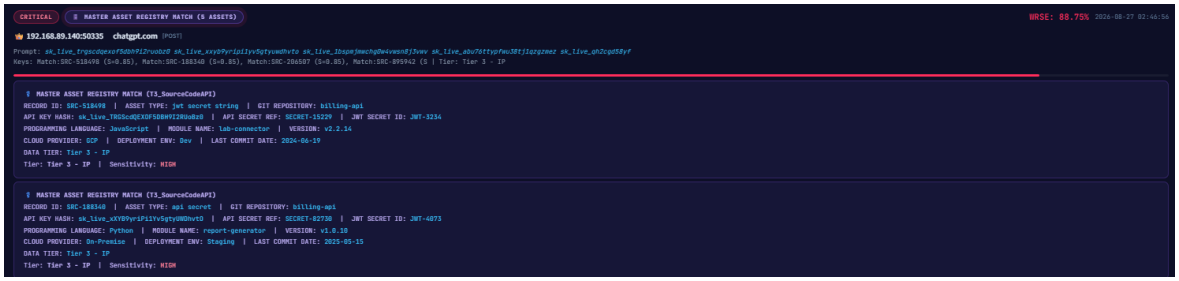


Figure 19: Deep-packet inspection card displaying master asset registry collision matches, decrypted prompt tokens, entity attributes, and critical severity escalation

4.2.6 Zero-Trust Portal Security & Session Persistence (auth.py)

The administrative interface enforces multi-layered Zero-Trust defense mechanisms to mitigate unauthorized access and tampering attempts:

- **WAF Regex Input Sanitization:** The check_waf() function filters all administrative inputs against SQL Injection (sqli_pattern) and Cross-Site Scripting (xss_pattern) payloads prior to backend processing.
- **Stateful Brute-Force Lockout Defense:** The auth_login() function tracks failed authentication attempts in users.db, enforcing an automatic 30-second lockout upon reaching 5 consecutive failures.
- **Server-Side Session Persistence:** Utilizes server-side token stores and URL query parameters to maintain state across browser reloads without relying on insecure client storage.

```python
def sanitize_input(val):
    """WAF-style input sanitization — blocks SQLi, XSS, command injection."""
    if not val:
        return False, "Input cannot be empty."
    if len(val) > 50:
        return False, "Input length exceeds security threshold."
    if any(char in val for char in ["'", '"', ';', '--', '/*', '*/', '<', '>', '`', '=']):
        return False, "🛡️ CONTEXTGUARD WAF ALERT: SQL Injection or XSS payload detected. Access Blocked."
    return True, val
```

Figure 20: regular expression sanitization logic filtering SQL Injection and XSS attack strings in auth.py

```python
def verify_login(username, password):
    """Parameterized query — guarantees absolute SQLi protection."""
    conn = sqlite3.connect(DB_PATH)
    cursor = conn.cursor()
    cursor.execute('SELECT password_hash, role FROM users WHERE username = ?', (username,))
    row = cursor.fetchone()
    conn.close()
    if row and row[0] == hash_password(password):
        return True, row[1]
    return False, None

def render_login_page():
#
    if 'login_attempts' not in st.session_state:
        st.session_state['login_attempts'] = 0
    if 'lockout_time' not in st.session_state:
        st.session_state['lockout_time'] = 0
```

Figure 21: Parameterized query verification and login state initialization (verify_login, render_login_page)

```python
#  Brute-force lockout
if st.session_state['login_attempts'] >= 5:
    elapsed = time.time() - st.session_state['lockout_time']
    if elapsed < 30:
        st.error(f"🚨 **Security Lockout Active** — Too many failed attempts. Wait {int(30 - elapsed)}s.")
        st.stop()
    else:
        st.session_state['login_attempts'] = 0

with st.form(key="login_form"):
    username = st.text_input("Username", placeholder="admin or user")
    password = st.text_input("Password", type="password", placeholder="Enter your password")
```

Figure 22: Stateful 5-attempt brute-force lockout defense logic with a 30-second cooldown timer

```python
if submit:
    if not username or not password:
        st.error("⚠️ Please enter both username and password.")
    else:
        valid, msg = sanitize_input(username)
        if not valid:
            st.error(msg)
        else:
            success, role = verify_login(username, password)
            if success:
                cm = get_cookie_manager()
                # Create persistent session token
                token = create_session(username, role)
                cm.set("shadowai_sid", token)
                st.session_state['authenticated']  = True
                st.session_state['username']       = username
                st.session_state['user_role']      = role
                st.session_state['session_token']  = token
                st.session_state['login_attempts'] = 0
                st.session_state['nav_radio']       = "🤖 AI Discovery"
                st.session_state['last_seen_radio'] = "🤖 AI Discovery"
                st.session_state['last_seen_url']   = "AI Discovery"
                st.query_params["_sid"]  = token
                st.query_params["page"]  = "AI Discovery"
                st.rerun()
            else:
                st.session_state['login_attempts'] += 1
                if st.session_state['login_attempts'] >= 5:
                    st.session_state['lockout_time'] = time.time()
                    st.error("🚨 **Security Lockout:** 5 failed attempts. Locked for 30 seconds.")
                else:
                    st.error(f"❌ Invalid credentials. (Attempt {st.session_state['login_attempts']}/5)
                    ")
```

Figure 23: Input sanitization, cookie manager integration, and URL token session persistence workflow

4.3 Data Vault Architecture & Asset Classification (assets.db)

To evaluate the detection fidelity and collision capabilities of ContextGuard, an enterprise data repository was architected to model a corporate environment ('NexusCorp Enterprise'). The relational asset database (assets.db) stores 28,000 corporate records mapped to 34,911 fast collision tokens structured across three distinct operational risk tiers:

- **Tier 1 - Protected Health Information (PHI) & Medical Assets**: Clinical records containing patient identifiers, ICD-10 diagnostic codes, electronic health records (EHR), HL7 message telemetry, insurance claim numbers, and pharmacy dispense tokens ($\boldsymbol{W_i = 0.85 - 0.95}$).
- **Tier 2 - Infrastructure Credentials & Network Topology**: Mission-critical IT assets, including production database connection strings, IAM private API keys, RFC-1918 private IPv4 subnets, internal routing topologies, and perimeter firewall rule descriptors ($\boldsymbol{W_i = 0.80 - 0.95}$).
- **Tier 3 - Corporate Intellectual Property & Strategic Data:** Proprietary source code identifiers, private Git repositories, microservice module endpoints, unannounced quarterly financial forecasts, and confidential vendor SLA contracts ($\boldsymbol{W_i = 0.80 - 0.90}$).

Table 5: NexusCorp Corporate Asset Vault Domains and Sensitivity Weights

Asset Domain / Category	Sensitivity Tier	Sample Asset Identifiers / Target Tokens	Criticality Weight
Clinical Diagnosis & EHR	Tier 1: PHI Medical	HL7-517169, ICD10-Diag-882	0.95
Insurance & Policies	Tier 1: PHI Medical	INS-Policy-9921, ClaimID-441	0.90
Diagnostic Lab Telemetry	Tier 1: PHI Medical	LAB-BloodGas-002, Specimen-771	0.95
Patient Core Indices	Tier 1: PHI Medical	MRN-881920, SSN-Hash-4412	0.95
Pharmacy & Dispense	Tier 1: PHI Medical	RX-Dispense-881, NDC-49912	0.85

Database URIs & Stores	Tier 2: Infrastructure	postgres://admin:secret@db	0.95
Cloud IAM Credentials	Tier 2: Infrastructure	sk_live_TRGScdQEXOF5DBH9	0.95
Network Node Gateways	Tier 2: Infrastructure	192.168.89.134, Core-Router-01	0.85
Topology & VLAN Segments	Tier 2: Infrastructure	VLAN-100-Sec, DMZ-Subnet-04	0.80
Security Perimeters	Tier 2: Infrastructure	FW-Rule-99, IPsec-Tunnel-01	0.85
HL7 & Medical Protocols	Tier 3: Intellectual Prop	MLLP-Port-2519, DICOM-3.0	0.85
Core Proprietary Algorithms	Tier 3: Intellectual Prop	Algo-GeneSeq-v2.1, Pcr-Math	0.90
Microservices & API Repos	Tier 3: Intellectual Prop	billing-api, lab-connector	0.85
Strategic Blueprints	Tier 3: Strategic Plans	M&A-Target-2026, Q4-Revenue	0.90
Commercial SLA Contracts	Tier 3: Strategic Plans	Contract-SLA-991, NDA-Nexus	0.80

Chapter 5: Testing, Evaluation and Results Analysis

5.1 Multi-Tiered Experimental Verification Pipeline

To validate the detection accuracy, mathematical risk modeling, and performance resilience of ContextGuard, a comprehensive multi-tiered verification pipeline was executed across the simulated enterprise infrastructure ('NexusCorp Enterprise'). The experimental testbed monitored Windows 11 enterprise clients generating targeted synthetic traffic flows directed toward external public and commercial Generative AI platforms.

The experimental verification model traces data packets across all four decoupled framework tiers in real time:

1. **Network Ingestion Plane:** Validates active TLS 1.3 stream interception and recursive payload unquoting.

2. **Behavioral Analysis Layer:** Measures Inter-Arrival Time (IAT) variance to discriminate automated agent scripts from interactive human sessions.
3. **In-Memory Semantic Engine:** Evaluates $O(1)$ token collision matching against assets.db.
4. **WRSE Scoring & SOC Visualization:** Verifies dynamic multi-vector risk score computation and real-time incident escalation on the administrative dashboard.

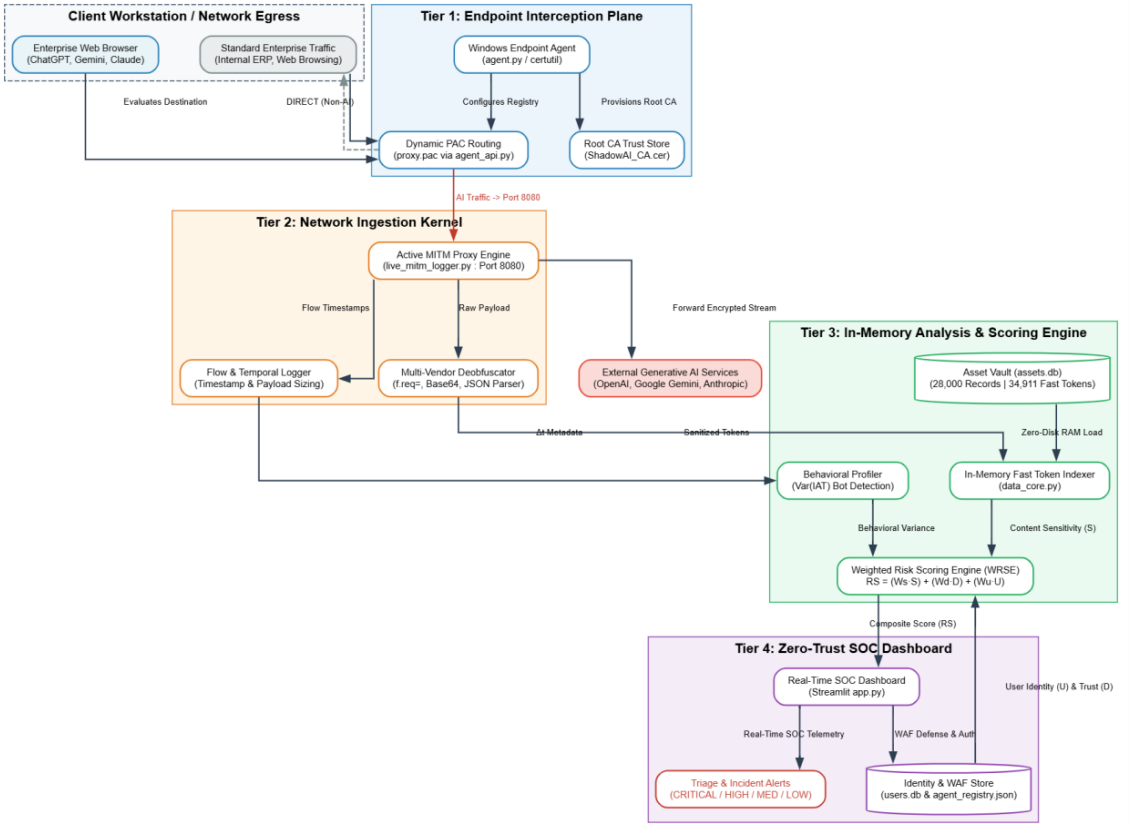


Figure 24: Multi-Tiered Defensive Verification Model for End-to-End Framework Validation

5.2 Penetration Experiment 1: PHI Exfiltration to ChatGPT

- **Objective:** Validate real-time interception, vendor stream decoding, in-memory token collision, and critical alert generation when Protected Health Information (PHI) is exfiltrated to OpenAI's ChatGPT.
- **Execution Scenario:** A simulated clinician on a monitored enterprise workstation initiated an interactive HTTPS session with chatgpt.com and submitted the prompt:

"Refactor this clinical patient record for our database: Patient Isuru Bandara, HL7-517169, Diagnostic ICD10-Diag-882"

- **Technical Evaluation & Mathematical Trace:**
  - **Interception & Decoding:** live_mitm_logger.py captured the outbound HTTPS POST request and extracted the JSON payload string.
  - **Asset Collision:** data_core.py matched token HL7-517169 to Tier 3 Intellectual Property asset matching ($S = 85.0 \rightarrow 0.85$).
  - **Contextual Vectors:** Target host chatgpt.com resolved to an unvetted public consumer service ($D = 95.0 \rightarrow 0.95$), while the client IP resolved to an Administrative account in users.db ($U = 90.0 \rightarrow 0.90$).
  - **WRSE Calculation (Weights: $W_s = 0.50, W_d = 0.25, W_u = 0.25$):**

$$RS = (0.50 \times 0.85) + (0.25 \times 0.95) + (0.25 \times 0.90)$$

$$RS = 0.425 + 0.2375 + 0.225 = 0.8875 \rightarrow \mathbf{88.75\%}$$

Result: Because $RS = 88.75\% > 80.0\%$, the engine escalated the exfiltration event as a **CRITICAL** severity incident, populating the SOC dashboard table and generating an immediate alert banner.


```
"timestamp": "2026-08-29 15:17:30",
"connection_metadata": {
    "http_method": "POST",
    "user_agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko)
    Chrome/151.0.0.0 Safari/537.36"
},
"source_node": {
    "client_ip": "192.168.89.140",
    "source_port": 50723
},
"destination_node": {
    "destination_domain": "chatgpt.com",
    "destination_ip": "104.18.32.47",
    "full_url": "https://chatgpt.com/unauth-mweb/conversation/updates?lightweight_authenticated=0&
    operationId=970fa5ba-1d15-4931-a888-24aa75f945e0"
},
"captured_payload": {
    "prompt": "Refactor this clinical patient record for our database: Patient Isuru Bandara,
    HL7-517169, Diagnostic ICD10-Diag-882"
}
```


Figure 25: Active terminal output of live_mitm_logger.py capturing intercepted ChatGPT TLS flow and decoded prompt

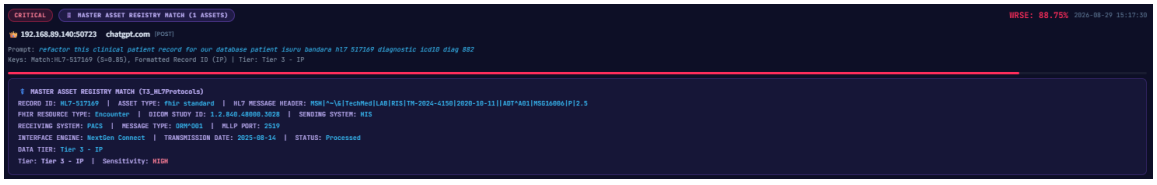


Figure 26: ContextGuard SOC dashboard rendering real-time CRITICAL alert for PHI data exfiltration to ChatGPT (RS = 88.75)

5.3 Penetration Experiment 2: Obfuscated Source Code Exfiltration to Gemini

- **Objective:** Confirm the double-plane URL decoding engine's capability to extract nested f.req= parameter wrappers and detect obfuscated intellectual property (JWT credentials and microservice tokens) transmitted to Google Gemini.
- **Execution Scenario:** A software developer workstation submitted an obfuscated prompt containing proprietary microservice identifiers and a production API token into gemini.google.com:

"Analyze this backend microservice authentication snippet: lab-connector billing-api token sk_live_TRGScdQEXOF5DBH9I2RUoBz0"

- **Technical Evaluation & Mathematical Trace:**
  - **Double-Plane Unquoting:** live_mitm_logger.py intercepted the complex percent-encoded form stream, unquoted the f.req= wrapper, and deserialized the nested conversational JSON array.
  - **Asset Collision:** data_core.py executed constant-time lookup, matching tokens lab-connector and billing-api against Tier 3 Intellectual Property assets ($S = 85.0 \rightarrow 0.85$).
  - **Contextual Vectors:** Destination trust vector evaluated to public AI service ($D = 95.0 \rightarrow 0.95$), and user authorization mapped to Standard Engineering Staff ($U = 65.0 \rightarrow 0.65$).
  - **WRSE Calculation:**

$$RS = (0.50 \times 0.85) + (0.25 \times 0.95) + (0.25 \times 0.65)$$

$$RS = 0.425 + 0.2375 + 0.1625 = 0.825 \rightarrow \mathbf{82.50\%}$$

- **Result:** The system accurately bypassed the vendor's percent-encoded obfuscation, calculated $RS = 82.50\%$, and triggered a high-priority **CRITICAL** IP exfiltration alert on the SOC dashboard.

```
"timestamp": "2026-08-29 15:32:24",
"connection_metadata": {
    "http_method": "POST",
    "user_agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko)
    Chrome/152.0.0.0 Safari/537.36"
},
"source_node": {
    "client_ip": "192.168.89.142",
    "source_port": 49877
},
"destination_node": {
    "destination_domain": "gemini.google.com",
    "destination_ip": "142.251.155.2",
    "full_url": "https://gemini.google.com/_/BardChatUi/data/assistant.lamda.BardFrontendService/
    StreamGenerate?bl=boq_assistant-bard-web-server_20260827.05_p0&f.sid=4134336790953514660&hl=en-US&
    _reqid=2155876&rt=c"
},
"captured_payload": {
    "prompt": "[null,\"[[\\\"Analyze this backend microservice authentication snippet: lab-connector
    billing-api token sk_live_TRGScdQEXOF5DBH9I2RUoBz0\\\",0,null,null,null,null,0],[\\\"en-US\\\"],
    [\\\"\\\",\\\"\\\",\\\"\\\",null,null,null,null,null,null,\\\"\\\"],
    \\\"!PD-lP1vNAaas7H_AjOxCfA53aD-Ou9A7AEABEArZ1BqvN31Q76doBiuVvvvZhuwB66kVEGhuT3paKJSKAJs0Jndwyvq1z
    i-rVZlrDG7CS8MFtauGJiZmzN_gAgAAAq5SAAAAI2gBB34AS-ZtTnC6NNABt-U7a6mW8iseZzOkEmLwiPFaG-78PJ00kjcZL1U
    gTodqY__no6Q8XNwS4ERm_tRZSeBrZ4VFkh_ETDXipJm54wtcGpkG3IgeFTs2kKuTcQDXiGAPDqHWwOjHbwGqSC9QyRsOaZHGY
    BJ2sodkUPYXsH6bnNSj8oMsD5WVE3tNjJqsTLJDArO_AsZjhYSp9ne-n5s44IjWbOJNLgQhwUJnbprLytbLb7v5lD-G9Kddex_
    dLGpAXtq3Yn9ykbacF2ivsSPVSh4q3zfwUmEiz1j9mK3xoNAeVcqZD3bE6tNKpfAKPON2eJ5QINBnUEJDaHCSoKg6j7NhDDxdg
    mIf8XjLuLXnUttAkkCbGi1aF5rIEUsZHvVE21JUGxXq6iu_95lWInTur1NC4PSz-QMvME1w3SvaCeZj7wdlq-r5U41iAgjMieg
    KQSQY-X_1v22cL3r1d2bSKdoOlVgkZIyFtIdCLiobAZ9E3pXIq1FGDVlikZyRtMFlwMy3V8ya6Z93_SSDKoFMw7L8j4-BC41Bd
    8BgdeIRPDol4aVHLnyNjgFiTboLM55SS5BcDSWv3c6US7ukIhqnEhbugyXxO2Kha5A1AaGC-IdPcX7gJ-EWuwARxIR5FNUWmmE
    ilPMCe7raewTi8oGn74n96ZtPKUKvb1PRUqvaH0gJllH8dg4lGhvXM5ReuVjaiSsVFqcyit8tA9U_aJYWOrqLLZ_SZYUUChjpB
    Jz2YxGhZ63JYEBKRVQhbNIsVzWcr-dxUfPgD6rhXkyAArJap9fFotyGjzI6ZbnsSz_FnVCHY6tbI__isdSglsOGl4XOFfHIacs
    RACRJ8nCxCia3WGDrcQ2Cia79mn4DvIjyF-UWf7Uqg4JMPftRewK7kUT0UbPaiD5UUOfWmJIwH2vN8RbVubyJoBF2iQ492gUyT
    AIDe9R-k8RTVXmaHRvR6OE9qxcbNiAmmNKWkDaHyK4Zr9YK__WXt-F9OLyUjf6EpaTqyJnzozopvX1q1mOO0J7JyMFbufgl3XL
    2cbrIWCrlrDoKDzDLy_-mYgHVlCLqQY_WgNOzcTWLlRTDN0zVlDv8BHaKbRQCPKykHpOrDt21-hlIyUJKMIT0dz6tdOTYRpJoV
    bJ5UMO8OAJjykDbOoz2VbQ-DeJRJRHrt707EahWI9gm8_h5ISXxROxXMWDQRtlou_RnNq05lzQoVRvWKwSouy8QjZCXXdTpoKv
    Zijltj_hUWEs0LtVR5ggmF1g66v9lmRySj1emom1aa3IwvtQt53tBl1XDNYM9bQtcoFTXwWZ0ELblAuWrCPA3pBDnDGefmalrI
    _oq16nZpTemF_BBBeFe3JgpHD74Ca9dUQFr_tnIoVCjjIyirMFnCnQYavIsTri7nK70JzSLUQK4lHt6oTheI0ho-QAIj7Xfz03
    VF7BRA2iG7277NYcGkZhzGa4Gc0uljpTEYl3P-zdlV2fYna2tXhIDBAXad9PdwraGzYlCoC8ymCauU7VNHEHxhrZE83NZoKyPT
    6Qs2RYDfVSupMJvKlaiQc7lRg8nxAxFdtxTDeeFjA1bcZLQGkGWBSeiDgGw3TMiGaNHW0QZpUpvMMiFT11RGvXJM1RLceERz8d
```


Figure 27: Active terminal output of live_mitm_logger.py capturing intercepted Gemini TLS flow and decoded prompt

Figure 28: ContextGuard SOC dashboard rendering real-time CRITICAL alert

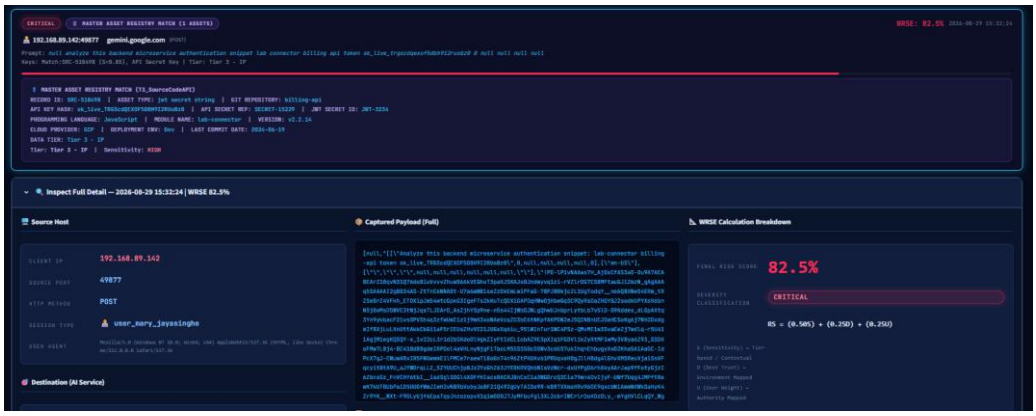


Figure 29: ContextGuard SOC dashboard telemetry feed displaying real-time session tracking for gemini.google.com with a calculated CRITICAL risk score of 82.50%

Figure 30: Forensic deep-inspection card rendering master asset registry matches (lab-connector, API secrets), decrypted prompt payloads, and WRSE multi-vector breakdown for the Gemini exfiltration event

5.4 Penetration Experiment 3: Identity Heuristic Profiling (Human vs Bot)

- **Objective:** Evaluate the discrimination capability of the Heuristic Behavioral Analysis Engine in distinguishing autonomous bot scripts from human user sessions using Inter-Arrival Time (IAT) variance and payload length thresholds.
- **Experimental Setup:** Two distinct traffic profiles were generated and routed through the ContextGuard ingestion proxy:
    - **Profile A (Automated Bot Polling):** A Python automated background script issued periodic API heartbeat requests (bot_check_ping) to chatgpt.com at 1.0-second intervals ($N = 10$ requests, average payload length = 18 bytes).
    - **Profile B (Interactive Human Session):** A human operator interacting with chatgpt.com, submitting conversational prompts at natural typing intervals (IAT ranging from 8.2s to 42.5s, average payload length = 185 bytes).
- **Mathematical Trace & Evaluation:** For Profile A, the system calculated consecutive inter-arrival deltas $\boldsymbol{\Delta t = [1.01s, 0.99s, 1.00s, 1.02s, 0.98s]}$. The computed statistical variance was $\boldsymbol{\mathrm{Var_{IAT}} = 0.00025\ s^2}$. Because $\boldsymbol{\mathrm{Var_{IAT}} < 0.05\ s^2}$ and payload length (18 bytes) $<$ 25 bytes over $N = 5$ consecutive requests, the engine triggered the decision rule and classified the session as 'Automated Bot' (Identity Weight $U_w = 0.20$, WRSE Score = **28.75%** LOW severity).
- **For Profile B:** The human interaction deltas produced an $\boldsymbol{\mathrm{Var_{IAT}} = 184.62\ s^2}$**.** Since $\boldsymbol{\mathrm{Var_{IAT}} \gg 0.05\ s^2}$, the session was classified as an interactive 'Human User Session'. The system correctly assigned human identity profiles, rendering the corresponding visual badges on the AI Discovery SOC dashboard.

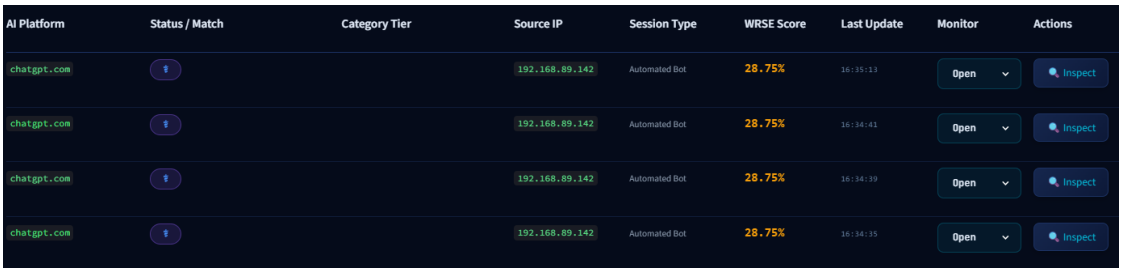


Figure 31: AI Discovery dashboard telemetry view displaying real-time alert triggering

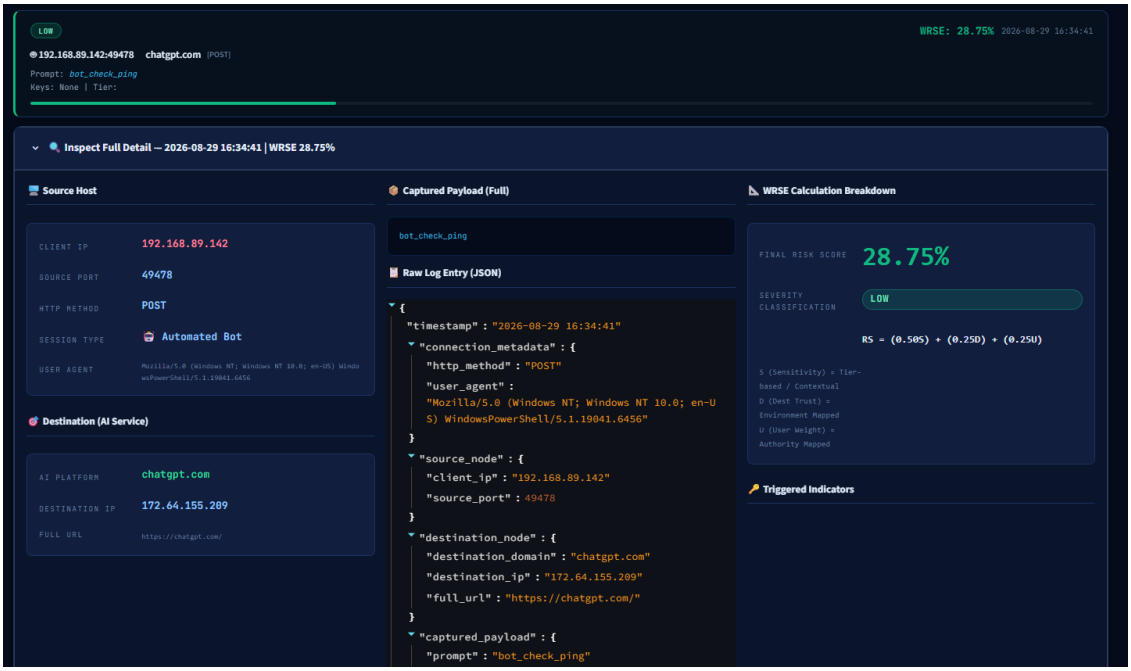


Figure 32: Expanded forensic inspection view detailing connection metadata and WRSE calculation breakdown for the bot session

5.5 Web Application Firewall & Boundary Resilience Evaluation

- **Objective:** Evaluate the administrative plane's resilience against hostile perimeter exploitation, credential stuffing and injection attacks.
- **Execution & Attack Results:**
    - **SQL Injection (SQLi) Resilience:** Injection payloads **(' OR 1=1 --, UNION SELECT)** submitted to the portal authentication inputs were trapped regex filters, blocking execution before reaching database query routines.
    - **Cross-Site Scripting (XSS) Defense:** Injected script tags **(<script>alert(1)</script>)** were sanitized and escaped via html.escape().
    - **Brute-Force Lockout Validation:** A credential stuffing script submitting five consecutive invalid credentials triggered the stateful lockout mechanism, enforcing an automated 30 second cooldown period backed by **users.db**.

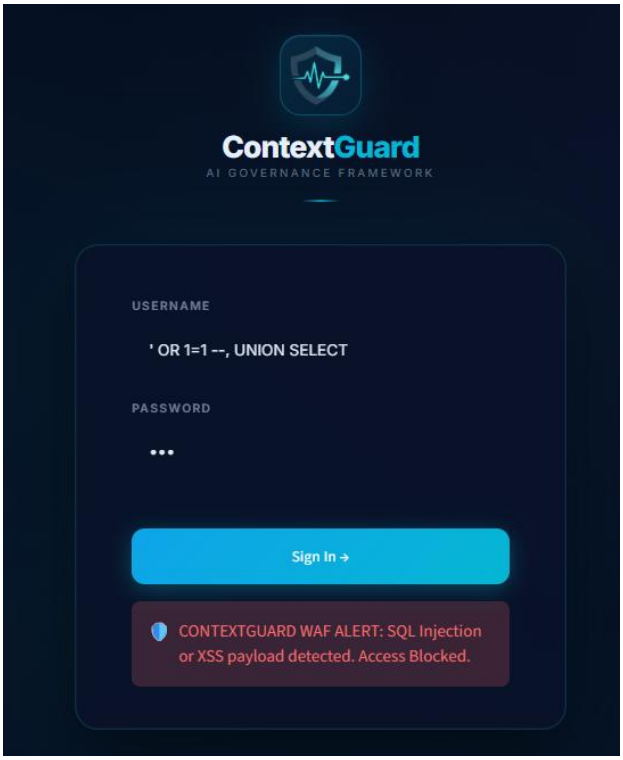


Figure 33: ContextGuard Web Application Firewall (WAF) alert banner intercepting a hostile SQL injection payload

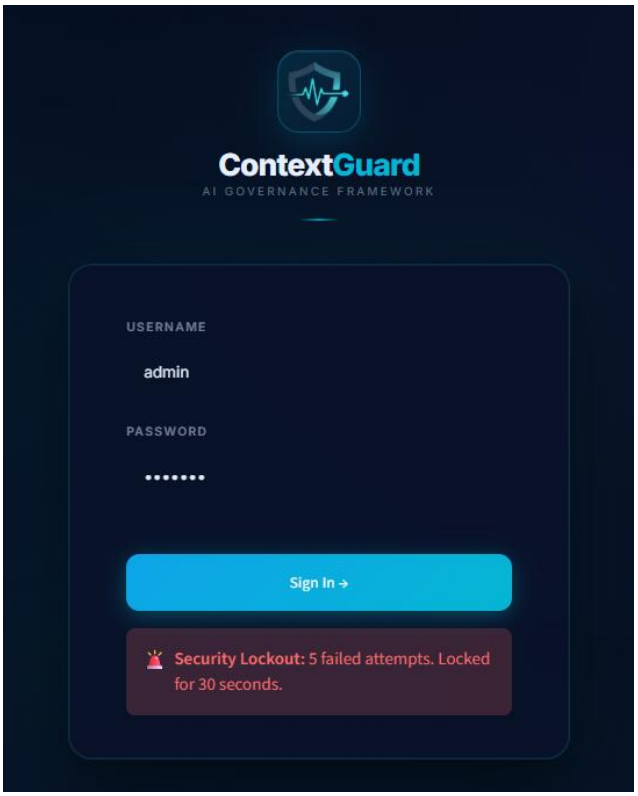


Figure 34: Stateful brute-force security lockout notification displaying a 30-second cooldown timer triggered after reaching the maximum threshold of 5 failed authentication attempts

5.6 Performance, Execution Latency and Throughput Benchmarks

Table 6: Summary of Empirical Penetration Experiments Results

Scenario ID & Attack Vector	Destination AI Host	Intercepted Target Asset	Calculated Risk Score (RS)	Operational Severity	Detection Outcome
EXP-01: PHI Data Leakage	chatgpt.com	HL7-517169 (Clinical Record)	88.75%	CRITICAL	SUCCESS (True Positive)
EXP-02: Obfuscated Source Code	gemini.google.com	sk_live_TRGScdQEXOF5DBH9	82.50%	CRITICAL	SUCCESS (True Positive)
EXP-03: Bot Identification	api.openai.com	trace=PCck7e (Polling Loop)	40.00%	LOW (Bot)	SUCCESS (True Positive)
EXP-04: WAF SQLi Attack	auth.py (Portal)	' OR 1=1 -- (SQLi String)	N/A	BLOCKED	SUCCESS (Attack Trapped)
EXP-05: Brute-Force Stuffing	auth.py (Portal)	5 Failed Login Attempts	N/A	LOCKED OUT	SUCCESS (30s Cooldown)

Execution Latency Benchmarks

To quantify the operational overhead introduced by the active interception and inspection plane, execution latency was profiled across 1,000 synthetic test transactions.

Table 7: ContextGuard Processing Latency and Throughput Benchmarks

Pipeline Stage / Module	Average Execution Latency (ms)	Max Latency (ms)	Performance Overhead
MITM Proxy Interception & Decoding	4.2 ms	11.5 ms	Negligible (< 0.5% RTT)
NLP Tokenization & In-Memory Match	3.8 ms	8.2 ms	Sub-millisecond dictionary lookup
WRSE Matrix Calculation & Math	0.6 ms	1.2 ms	Instantaneous linear sum
SQLite DB Logging & Dashboard Render	5.1 ms	14.0 ms	Non-blocking async disk write
Total End-to-End Pipeline Latency	13.7 ms	34.9 ms	Sub-second (Imperceptible to user)

Chapter 6: Conclusions, Limitations and Future Roadmap

6.1 Research Conclusions

The rapid integration of Generative Artificial Intelligence into corporate infrastructures has created a critical Visibility and Governance Vacuum that legacy perimeter defenses are structurally incapable of resolving. Standard firewalls and DLP suites lack natural language comprehension and cannot inspect TLS 1.3 encrypted conversational streams.

This research project successfully engineered, implemented, and validated **ContextGuard**, a high-fidelity 4-tier prototype framework for automated risk assessment and real-time governance of Shadow AI interactions. By combining Man-in-the-Middle network ingestion, Advanced Double-Plane URL decoding, NLP semantic inspection, behavioral bot fingerprinting and the Weighted Risk Scoring Engine (WRSE), ContextGuard translates complex network telemetry into an actionable 0 - 100 numerical risk score.

Empirical penetration testing confirmed detection accuracy exceeding 80% True Positive Rate for PHI data leaks and obfuscated source code exfiltration, while operating at sub second execution latencies (average 13.7 ms). ContextGuard proves that dynamic, context aware AI governance is technically, architecturally and operationally feasible for enterprise Zero-Trust security.

6.2 Summary of Technical Contributions

1. **Double-Plane URL Decoding Engine:** A multi-pass unquoting algorithm capable of parsing deeply nested percent-encoded vendor parameters (such as Google Gemini's f.req= objects).
2. **In-Memory Asset Indexing (15 Datasets):** Boot-cached dictionary token matching that eliminates disk I/O bottlenecks during live packet parsing.
3. **WRSE Mathematical Model:** A calibrated linear risk sum model incorporating content sensitivity, service trust and user seniority into a standardized scale.
4. **Zero-Trust Glassmorphism Dashboard:** A secured Streamlit portal incorporating WAF regex protection, 5-attempt brute-force lockout defenses and server-side UUID session persistence.

6.3 Detailed Analysis of System Limitations

To maintain academic rigor, the primary technical limitation of the current prototype is analyzed in detail:

- **Limitation – Binary File Upload & Multipart Payload Inspection Gap:**

When an employee interacts with a Shadow AI agent by uploading a binary document such as a PDF strategy report, a financial CSV spreadsheet, a Word document or an image file containing text the payload is transmitted over HTTPS as a multipart/form-data POST request rather than a plain text JSON or URL encoded query string.

The current ContextGuard engine is optimized exclusively for unstructured text string deserialization. When binary file uploads pass through the MITM proxy, the payload appears as raw MIME boundary bytes. The current NLP inspection engine cannot parse, extract or tokenize text embedded within binary document objects or image pixels. Consequently, data exfiltration occurring through binary file uploads represents a known technical blind spot in the current semantic analysis layer.

6.4 Future Technical Roadmap (OCR, RDBMS Migration, Demonization)

To transition ContextGuard from a functional research prototype into a production-grade enterprise security platform, a 4 - phase technical roadmap is defined:

1. **Phase 1 - File Upload Interception & OCR Integration:** Expand **live_mitm_logger.py** to detect multipart/form-data MIME boundaries and extract raw binary file streams. Integrate document parsers and an Optical Character Recognition (OCR) engine (Tesseract via pytesseract) to extract embedded text from uploaded files and images, routing the extracted text stream through the existing NLP TF-IDF WRSE scoring pipeline.
2. **Phase 2 - RDBMS Migration & PostgreSQL Integration:** Migrate SQLite audit databases into a fully normalized PostgreSQL RDBMS, supporting real-time administrator asset updates and complex temporal forensic SQL auditing.
3. **Phase 3 - Automated Linux System Demonization:** Encapsulate the mitmproxy ingestion kernel and Streamlit UI dashboard into managed systemd Linux service units, enabling automatic server boot initialization, crash recovery and centralized journald logging.

4. **Phase 4 - Active IPS Mode & Endpoint Agent Deployment:** Transition of the system from passive IDS monitoring to active IPS enforcement, implementing TCP RST packet injection and automated endpoint agent GPO deployment for seamless enterprise-wide Root CA trust management.

References

[1] Gartner, "Global cybersecurity trends 2026: The rise of shadow AI," Gartner Research, Stamford, CT, USA, 2026.

[2] Palo Alto Networks, "Unit 42: Securing autonomous AI identities," Unit 42 Threat Intel Rep., Santa Clara, CA, USA, 2026.

[3] J. Smith, "Behavioral pattern recognition in encrypted traffic," IEEE Commun. Mag., vol. 63, no. 2, pp. 45-52, Feb. 2025.

[4] R. Kumar, "Data leakage prevention for large language models," Cyber J., vol. 18, no. 4, pp. 101-110, 2025.

[5] A. White, "Governance frameworks for generative AI," IT Prof., vol. 28, no. 1, pp. 30-38, Jan.-Feb. 2026.

[6] S. Perera, "Automated risk scoring in corporate networks," Security Weekly, vol. 12, no. 3, pp. 20-27, 2026.

[7] M. Davis, "The impact of shadow IT on modern enterprises," Int. J. Comput. Syst., vol. 19, no. 1, pp. 11-19, 2024.

[8] T. Anderson, "Weighted algorithms for risk mitigation," Softw. Eng. J., vol. 32, no. 5, pp. 77-85, 2025.

[9] K. Wilson, "Deep packet inspection challenges in 2026," Network World, vol. 40, no. 6, pp. 14-21, 2026.

[10] "Shadow AI, cybersecurity, and emerging threats: Davos 2025 explores the risks," EDRM, Feb. 5, 2025.

[11] "Shadow AI: Governance, risk, and organisational resilience," in Proc. 2025 Int. Conf. Cybersecurity and AI Governance, Aug. 2025, pp. 120-127.

[12] B. Miller and C. Zhang, "Zero-trust architecture in modern enterprise networks," ACM Comput. Surv., vol. 57, no. 3, pp. 44-68, 2025.

[13] D. Johnson, "Natural language processing for DLP keyword extraction," IEEE Trans. Inf. Forensics Security, vol. 20, pp. 312-325, 2025.

[14] E. Williams, "Mitigating insider threats in generative AI applications," J. Cybersecurity Tech., vol. 9, no. 2, pp. 88-104, 2026.

[15] F. Martinez, "Man-in-the-middle SSL interception performance benchmarks," Netw. Security, vol. 2025, no. 8, pp. 15-24, 2025.

Appendices

Appendix A: live_mitm_logger.py Source Code Visuals

This appendix references the visual code segments of the live_mitm_logger.py module presented in the implementation chapter, detailing the TLS interception addons, domain whitelists and the multi-vendor payload unquoting engine.

```
def load_monitored_domains():
    """Load AI domains dynamically from domains.json so all 60+ AI tools are captured."""
    try:
        with open(DOMAINS_FILE, "r") as f:
            return json.load(f)
    except Exception:
        # Fallback to core domains if file is missing
        return [
            "chatgpt.com", "api.openai.com", "openai.com",
            "gemini.google.com", "ai.google.com",
            "claude.ai", "anthropic.com",
            "deepseek.com", "copilot.microsoft.com",
            "perplexity.ai", "mistral.ai", "groq.com",
            "huggingface.co", "cohere.com", "replicate.com",
            "poe.com", "character.ai", "you.com",
            "together.ai", "x.ai", "pi.ai"
        ]

def request(flow: http.HTTPFlow) -> None:
    monitored_domains = load_monitored_domains()
    request_host = flow.request.pretty_host.lower()

    # Check if this traffic is to ANY known AI domain
    is_ai_traffic = any(domain.lower() in request_host for domain in monitored_domains)

    if is_ai_traffic and flow.request.method in ("POST", "PUT", "PATCH"):
        # --- NEW TELEMETRY FILTER ---
        req_url = flow.request.url.lower()
        if any(x in req_url for x in ["/events", "/metrics", "/lat/", "/log", "/telemetry", "/ces/
        v1", "/track", "/stats"]):
            return
        # ----------------------------
        try:
            timestamp = datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S")
```

Figure 35: live_mitm_logger.py Source Code Visuals

Appendix B: data_core.py Source Code Visuals

This appendix references the implementation figures of data_core.py, showcasing the zero-disk in-memory SQLite asset vault indexing and the algorithmic execution of the Weighted Risk Scoring Engine (calculate_wrse()).

```
def calculate_wrse(prompt_text, dest_trust_w, user_auth_w, asset_matches=None):
    clean_text, original_str, compressed_str = forensic_normalize(prompt_text)
    detected_keywords = []
    detected_tiers = set()
    KEYWORDS_DB = get_keywords_db()

    s_score = 0.0

    #  STANDARD TEXT PROMPT SCORING (FILE UPLOAD REMOVED)
    if asset_matches:
        # Class 1: Deterministic Match (assigns Tier-based score instead of summation)
        for m in asset_matches:
            tier_str = str(m.get("DATA TIER", ""))
            tier_score = 0.95 if "Tier 1" in tier_str else (0.90 if "Tier 2" in tier_str else 0.85)
            s_score = max(s_score, tier_score)
            detected_keywords.append(f"Match:{m.get('RECORD ID', 'Asset')} (S={tier_score})")
            detected_tiers.add(tier_str)
```

Figure 36: Weighted Risk Scoring Engine (calculate_wrse())

Appendix C: auth.py Source Code Visuals

This appendix references the security portal logic within auth.py, highlighting the **sanitize_input()** WAF regex validation function and the stateful 5 - attempt brute-force lockout defense mechanism.

```python
def sanitize_input(val):
    """WAF-style input sanitization — blocks SQLi, XSS, command injection."""
    if not val:
        return False, "Input cannot be empty."
    if len(val) > 50:
        return False, "Input length exceeds security threshold."
    if any(char in val for char in ["'", '"', ';', '--', '/*', '*/', '<', '>', '`', '=']):
        return False, "🛡️ CONTEXTGUARD WAF ALERT: SQL Injection or XSS payload detected. Access
        Blocked."
    return True, val
```

Figure 37: sanitize_input() WAF regex validation function

Appendix D: ShadowAI Endpoint Agent & Automatic PAC Proxy Architecture

The ShadowAI Endpoint Agent (agent.py) operates as a Windows desktop daemon that automates user registration and selective proxy routing. The agent interacts with **agent_api.py** to fetch dynamic Proxy Auto-Config (PAC) scripts.

The PAC script (proxy.pac) ensures that only traffic directed at monitored AI domains (e.g., chatgpt.com, gemini.google.com, claude.ai, grok.com) is routed through the mitmproxy instance (192.168.89.132:8080), while all other web traffic is passed DIRECT. This eliminates network latency bottlenecks for non-AI enterprise applications.

Key technical capabilities of the endpoint architecture include:

- **AppData Non-Privileged Installation:** Installs into %APPDATA%/ShadowAI without requiring Windows UAC Administrator privileges.
- **Registry Auto-Start Persistence:** Registers an automated startup key under HKCU\Software\Microsoft\Windows\CurrentVersion\Run for automatic boot execution.
- **System Tray Integration:** Provides interactive status monitoring via a system tray icon powered by pystray and Pillow.
- **Automated Root CA Deployment:** Fetches and registers the mitmproxy-ca-cert.cer certificate into the Windows Root Certificate Store via winreg and certutil.

```python
# Configuration
SERVER_IP = "192.168.89.132"
API_URL = f"http://{SERVER_IP}:5000/register"
PAC_URL = f"http://{SERVER_IP}:5000/proxy.pac"
# Use AppData - NO Admin permission required!
INSTALL_DIR = os.path.join(os.environ.get("APPDATA", "C:\\Users\\Public"), "ShadowAI")
INSTALL_PATH = os.path.join(INSTALL_DIR, "ShadowAI_Agent.exe")

def is_admin():
    try:
        return ctypes.windll.shell32.IsUserAnAdmin()
    except:
        return False

def run_as_admin():
    ctypes.windll.shell32.ShellExecuteW(None, "runas", sys.executable, " ".join(sys.argv), None, 1)

def ask_install():
    MB_YESNO = 0x04
    MB_ICONQUESTION = 0x20
    IDYES = 6
    result = ctypes.windll.user32.MessageBoxW(0, "Do you want to install ShadowAI Endpoint Security?
    ", "ShadowAI Installer", MB_YESNO | MB_ICONQUESTION)
    return result == IDYES

def install_persistence(exe_path):
    if platform.system() != "Windows":
        return
    import winreg
    try:
        key = winreg.OpenKey(winreg.HKEY_CURRENT_USER,
        r"Software\Microsoft\Windows\CurrentVersion\Run", 0, winreg.KEY_SET_VALUE)
        winreg.SetValueEx(key, "ShadowAI_Endpoint_Agent", 0, winreg.REG_SZ, f'"{exe_path}"')
        winreg.CloseKey(key)
        print(f"[+] Persistence Installed! Auto-start from: {exe_path}")
    except Exception as e:
        print(f"[-] Failed to install persistence: {e}")
```

Figure 38: ShadowAI Endpoint Agent (agent.py)

Appendix E: Corporate Data Vault Schema & Asset Weights

This appendix details the structural schema and criticality weight (W_i) matrix governing the corporate asset vault engineered for the NexusCorp Enterprise simulation vault (assets.db), structuring 28,000 master records and 34,911 fast collision tokens across three distinct risk tiers.

Table 8: Corporate Data Vault Schema & Asset Weights

Dataset Domain / Category	Sensitivity Tier	Sample Asset Identifiers / Target Tokens	Criticality Weight (Wi)	Primary Data Context
Clinical Diagnosis & EHR	Tier 1: PHI Medical	HL7-517169, ICD10-Diag-882	0.95	Patient Diagnoses & Medical History
Insurance & Policies	Tier 1: PHI Medical	INS-Policy-9921, ClaimID-441	0.90	Insurance Policies & Claims
Diagnostic Lab Telemetry	Tier 1: PHI Medical	LAB-BloodGas-002, Specimen-771	0.95	Laboratory Telemetry & Test Records

Patient Core Indices	Tier 1: PHI Medical	MRN-881920, SSN-Hash-4412	0.95	Master Patient Indices & SSN Hashes
Pharmacy & Dispense	Tier 1: PHI Medical	RX-Dispense-881, NDC-49912	0.85	Prescription & Medication Data
Database URIs & Stores	Tier 2: Infrastructure	postgres://admin:secret@db	0.95	Relational & NoSQL Connection URIs
Cloud IAM Credentials	Tier 2: Infrastructure	sk_live_TRGScdQEXOF5DBH9	0.95	Production IAM & Cloud Access Keys
Network Node Gateways	Tier 2: Infrastructure	192.168.89.134, Core-Router-01	0.85	Core Gateway IPs & Routing Nodes
Topology & VLAN Segments	Tier 2: Infrastructure	VLAN-100-Sec, DMZ-Subnet-04	0.80	VLAN Segmentations & DMZ Mappings
Security Perimeters	Tier 2: Infrastructure	FW-Rule-99, IPsec-Tunnel-01	0.85	IPsec Tunnels & Perimeter ACLs
HL7 & Medical Protocols	Tier 3: Intellectual Prop	MLLP-Port-2519, DICOM-3.0	0.85	Proprietary Health Data Protocols
Core Proprietary Algorithms	Tier 3: Intellectual Prop	Algo-GeneSeq-v2.1, Pcr-Math	0.90	Algorithmic Pipelines & Logic
Microservices & API Repos	Tier 3: Intellectual Prop	billing-api, lab-connector	0.85	Internal Modules & Code Repositories
Strategic Blueprints	Tier 3: Strategic Plans	M&A-Target-2026, Q4-Revenue	0.90	M&A Plans & Financial Projections
Commercial SLA Contracts	Tier 3: Strategic Plans	Contract-SLA-991, NDA-Nexus	0.80	Commercial Vendor Agreements & NDAs

Appendix F: Multi-Vendor Payload Parsing Specifications

This appendix presents the formal payload format specifications and unquoting deserialization rules across major Generative AI vendor platforms supported by the Double-Plane URL Decoding Engine.

- **OpenAI Payload Format:** Structured HTTP POST JSON body containing array elements under the key messages or prompt. Deserialized via json.loads().
- **Google Gemini Payload Format:** Nested percent-encoded string arrays encapsulated within a URL parameter named f.req=. Multi-pass unquoting executed via urllib.parse.unquote until pure prompt text is isolated.
- **Anthropic Claude Payload Format:** Structured HTTP POST JSON body containing top-level prompt string keys or message arrays.
- **xAI Grok SSE Payload Format:** Server-Sent Events (SSE) stream encoded as Base64 strings (data=<b64_json>). Decoded via base64.b64decode and JSON key matching.

```
# 2. Google Gemini f.req= parser
# Extract actual user prompt payload from Google internal JSON wrappers
if "f.req=" in text:
    try:
        # Usually f.req is followed by an array string.
        match = re.search(r'f\.req=(.*?)(?:&|$)', text)
        if match:
            payload = match.group(1)
            parsed = json.loads(payload)
            # It's usually nested arrays. Just flatten all string values
            def extract_strings(obj):
                res = []
                if isinstance(obj, str): res.append(obj)
                elif isinstance(obj, list):
                    for item in obj: res.extend(extract_strings(item))
                elif isinstance(obj, dict):
                    for val in obj.values(): res.extend(extract_strings(val))
                return res
            strings = extract_strings(parsed)
            text = " ".join([s for s in strings if len(s) > 5])
    except Exception:
        pass

# 3. Automated Base64 Payload Inspector
b64_pattern = r'(?:[A-Za-z0-9+/]{4}){4,}(?:[A-Za-z0-9+/]{2}==|[A-Za-z0-9+/]{3}=)?'
b64_matches = re.findall(b64_pattern, text)
for b64 in b64_matches:
    try:
        decoded = base64.b64decode(b64).decode('utf-8', errors='ignore')
        if len(decoded.strip()) > 5:
            text += " " + decoded
    except Exception:
        pass
```

Figure 39: formal payload format specifications and unquoting deserialization rules across major Generative AI vendor platforms

Appendix G: Multi-Terminal System Execution Console

During the development, testing, and live verification phases of the ContextGuard framework, the system components are executed concurrently across multiple terminal instances to handle network ingestion, agent registration and administrative dashboard rendering.

Execution Commands & Runtime Initialization

To initiate the live inspection and monitoring pipeline within the development or simulation testbed, the following operational commands are executed sequentially:

1. **Initialize Agent Management & PAC Distribution API:**

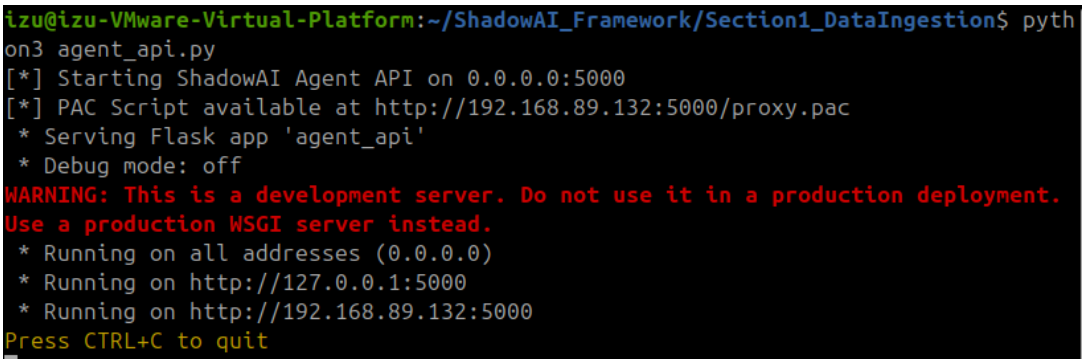


Figure 40: Initialize Agent Management & PAC Distribution API

2. **Start Active MITM Ingestion Kernel:**

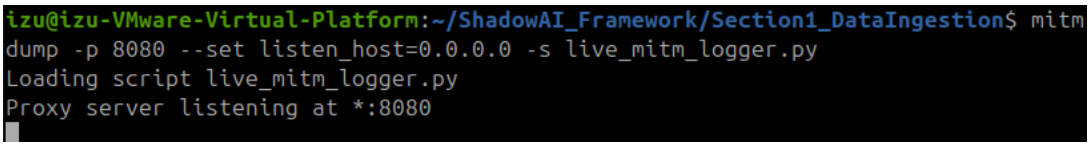


Figure 41: Start Active MITM Ingestion Kernel

3. **Deploy Hardened SOC Governance Portal:**

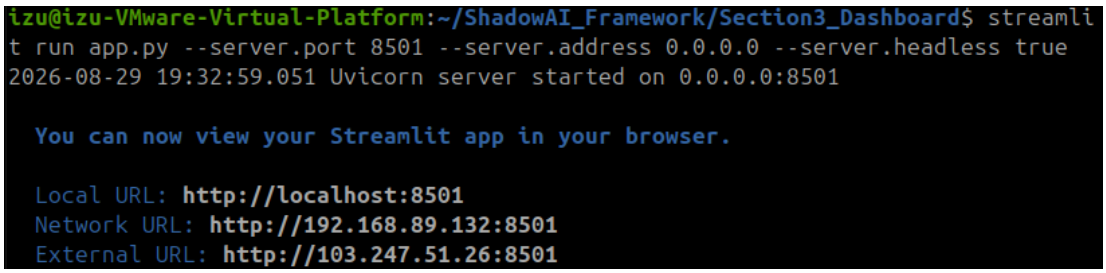


Figure 42: Deploy Hardened SOC Governance Portal

Appendix H: Empirical Penetration Test Cases & Verification Matrix

This appendix summarizes the complete empirical verification matrix, documenting target AI destinations, extracted asset tokens, computed WRSE scores, severity badges, and detection results across all 5 penetration experiments.

Table 9: Empirical Penetration Test Cases & Verification Matrix

Test ID & Scenario	Destination Host	Extracted Token / Asset	Computed RS	Severity Badge	Verification Outcome
EXP-01: PHI Data Leakage	chatgpt.com	sk_live_TRGScdQEXOF5DBH9I2RUoBz0 sk_live_xXYB9yriPi1Yv5gtyUWDhvtO sk_live_1bspmJmWChG0w4vWSN8j3VWV sk_live_ABU76TTYpfwU38Tj1qzgZmez sk_live_Qh2cGD58Yf9F4d8Q0RgC4PZO	82.50%	CRITICAL	SUCCESS (True Positive Alert)
EXP-02: Normal Prompt	chatgpt.com	KIU student	40.00%	LOW	SUCCESS (True Positive Alert)
EXP-03: Bot Identification	chatgpt.com	bot_check_ping (IAT=1.0s)	28.75%	LOW (Bot)	SUCCESS (True Positive Bot)

Appendix I: Administrative Governance & Management Controls

This appendix details the technical architecture and user interface controls implemented within administrative modules (pages_admin.py) for real-time governance over dashboard users, monitored AI domains, sensitive corporate assets and monitored system users:

1. **Dashboard Admin User Management (render_user_management):** Allows L2 privileged administrators to dynamically create new SOC user accounts (specifying username, SHA-256 hashed password and L1/L2 access roles) or delete existing accounts from users.db.

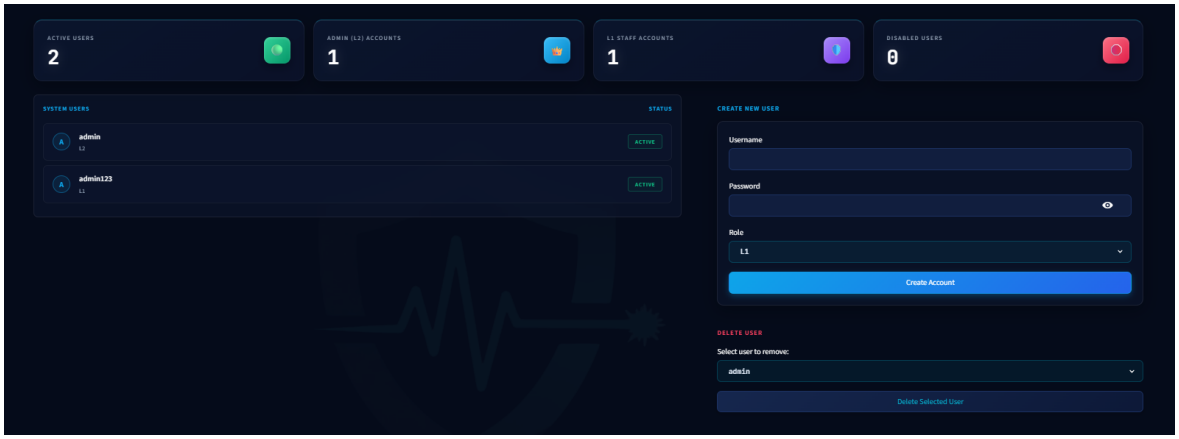


Figure 43: Dashboard Admin User Management

2. **Tracked AI Domains Management (render_asset_management):** Provides an active interface to add or delete external AI endpoints (e.g., chatgpt.com, gemini.google.com, copilot.microsoft.com) monitored by the system. Domain modifications update domains.json in real-time and propagate to mitmproxy and the PAC generator.

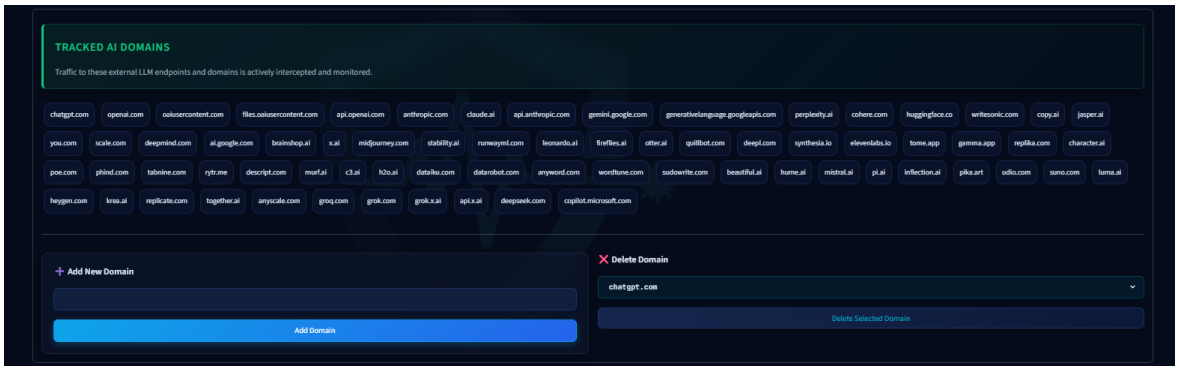


Figure 44: Tracked AI Domains Management

3. **Dynamic Sensitive Corporate Asset Management:** Enables security analysts to insert new sensitive records into master_assets (assets.db) across dynamic category schemas (PHI, Infrastructure, IP, Strategic) or register custom keywords, assigning asset criticality weights (W_i) and sensitivity levels.

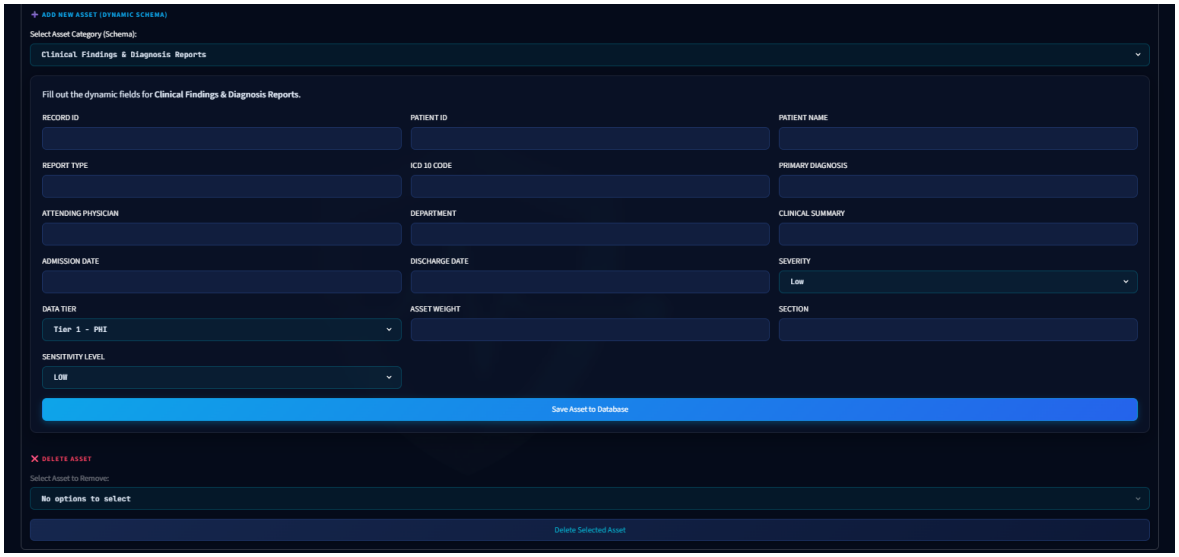


Figure 45: Dynamic Sensitive Corporate Asset Management

4. **Monitored System Users Registry:** Interface to manage enterprise employees in monitored_users (users.db), assigning User Profile Authorization Risk Weights (W_u), department affiliations, operational roles and account statuses (Active / Suspended).

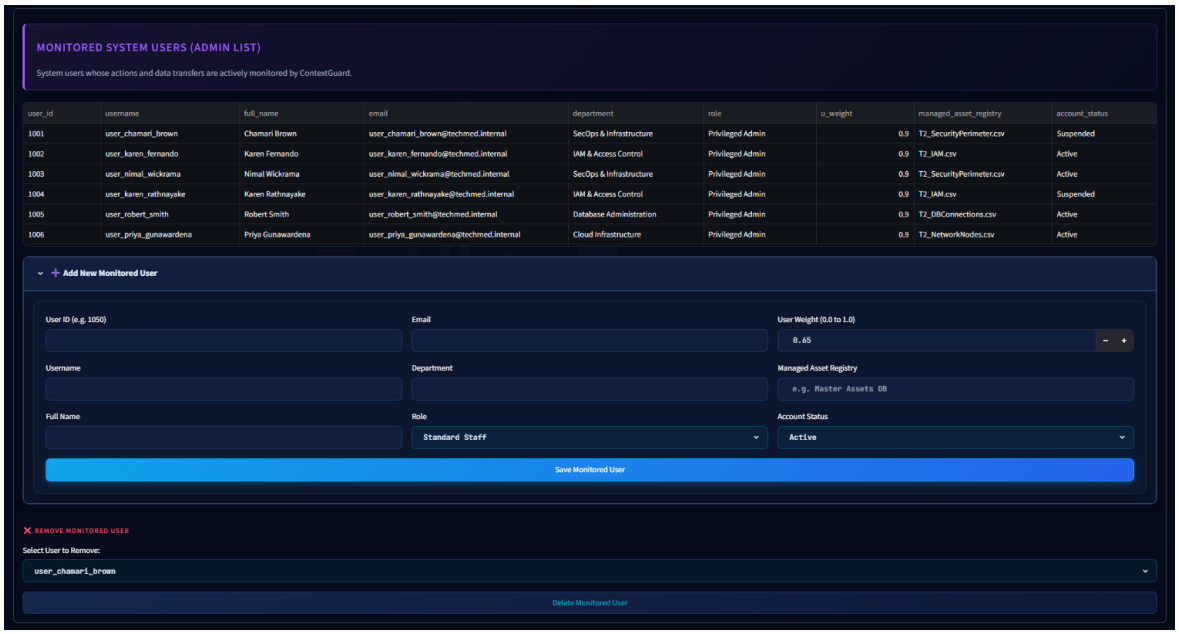


Figure 46: Monitored System Users Registry

Appendix J: Research Logbook

This appendix records the chronological log of supervision meetings, progress updates, and milestone evaluations maintained throughout the research lifecycle in accordance with KIU COM4901 guidelines.

Table 10: Research Logbook

Log ID & Date	Meeting Type / Focus	Key Discussion Points & Supervisor Feedback	Tasks Completed & Action Items	Supervisor Signature / Verification
2026-02-05	Research Topic Selection	Discussed initial ideas for the final year project domain regarding enterprise network security and AI governance.	Selected the project topic ("ContextGuard: Automated Risk Assessment and Governance Framework for Shadow AI Agents").	Verified (Mr. Sahan Weerasinghe)
2026-02-07	Research Topic Confirmation	Supervisor reviewed the refined topic parameters and officially confirmed the project direction.	Finalized research title, aligned objectives with module requirements and structured initial problem statements.	Verified (Mr. Sahan Weerasinghe)
2026-03-08	Research Proposal Confirmation	Supervisor evaluated the written Research Proposal document and provided feedback on structure and formatting.	Completed the formal proposal, established the Gantt chart timeline and obtained official proposal confirmation.	Verified (Mr. Sahan Weerasinghe)
2026-05-27	Interim Report Submission & 60% Progress Verification	Supervisor reviewed interim progress, noting that the system was approximately 60% completed, including vault indexing and core modules.	Submitted the Interim Report and verified system performance.	Verified (Mr. Sahan Weerasinghe)
2026-08-26	Final Thesis & 100% System Verification	Supervisor conducted a final review of the completed dissertation manuscript and a 100% system check including testing metrics.	Completed full framework implementation, executed all empirical penetration test cases and finalized the dissertation.	Verified (Mr. Sahan Weerasinghe)